UNIT-I

Introduction To Gen Ai: Historical Overview of Generative modelling, Difference between Gen AI and Discriminative Modelling, Importance of generative models in AI and Machine Learning, Types of Generative models, GANs, VAEs, autoregressive models and Vector quantized Diffusion models, understanding if probabilistic modelling and generative process, Challenges of Generative Modelling, Future of Gen AI, Ethical Aspects of AI, Responsible AI, Use Cases.

Introduction To Gen AI

Generative Artificial Intelligence (Generative AI) refers to a class of AI models that are designed to learn the underlying patterns and structure of input data in order to generate new, original content that is statistically similar to the training data. These models typically estimate the joint probability distribution of inputs and can produce novel outputs such as text, images, audio, video, or code.

(or)

Generative AI refers to **artificial intelligence models that can create new content**, such as text, images, music, code, or even video, by learning patterns from existing data.

These models don't just analyse data — they **generate** something **original** that resembles what they've been trained on.

Examples :

- Write an essay or story (e.g., ChatGPT)
- Create a digital painting (e.g., DALL·E, Midjourney)
- Compose a song or melody
- Generate realistic human speech (e.g., voice cloning)
- Auto-generate code or documentation

Historical Overview of Generative Modelling

1950s–1980s: Foundations in Statistics

Early generative models such as Gaussian Mixture Models (GMMs) and Naive Bayes were developed.
These models aimed to learn the joint probability distribution of input data and output labels.

1990s–2000s: Probabilistic Graphical Models

Development of models like Hidden Markov Models (HMMs), Bayesian Networks, and Latent Dirichlet Allocation (LDA).
These provided more structured ways to model uncertainty and dependencies in data.

2014: Breakthrough with Generative Adversarial Networks (GANs) Ian Goodfellow introduced Generative Adversarial Networks (GANs).

GANs use a generator and a discriminator in a competitive setup to create highly realistic synthetic data, especially images.

This innovation marked a major turning point in the field of generative AI.

2017–2020: Deep Generative Models Expand

Introduction and growth of models such as Variational Autoencoders (VAEs), Pixel CNN, and Transformer-based models.

These models expanded generative capabilities across **text, images, audio, and beyond**.

2020s–Present: Generative AI Goes Mainstream

The rise of large-scale models like GPT-3, GPT-4, DALL·E, Stable Diffusion, and ChatGPT.

Generative AI becomes widely used in fields such as education, healthcare, design, entertainment, and programming.

Difference between the Importance of generative models in AI and Machine Learning

Aspect	Generative AI (Generative Modelling)	Discriminative Modelling
Definition	Models that learn the **joint probability** P(x, y) or P(x)P(x)	Models that learn the **conditional probability** P(y)
Purpose	Generate new data similar to the training data	Classify or predict labels from input data
What it learns	Both how data is distributed and how labels relate to features	Direct relationship between input features and output labels
Data Generation	Can **generate** realistic new samples (e.g., images, text, music)	Cannot generate new data
Examples	GANs, VAEs, Diffusion Models, Naive Bayes, GPT	Logistic Regression, SVM, Decision Trees, Random Forest, Neural Networks
Output	Synthetic data (e.g., images, text, audio)	Class labels, predictions
Use Cases	Text generation, image synthesis, creative AI, data augmentation	Classification, regression, and object detection
Training Complexity	Usually more complex and computationally intensive	Comparatively simpler and faster to train
Interpretability	Often less interpretable	More interpretable in many traditional models
AI Focus	Focuses on **creation** and **imagination**	Focuses on **decision-making** and **label prediction**

Importance of Generative Models in AI & Machine Learning

1. Learning Data Distribution

- Generative models learn how data is distributed, capturing the structure and patterns in datasets.
- This understanding is useful for building smarter systems that can reason about or mimic real data.

2. Data Generation

- They can generate new, realistic data samples, useful for training, testing, or creative applications.
- For example, generating images, text, or even tabular data that resembles real-world inputs.

3. Handling Limited or Unlabeled Data

- Useful in semi-supervised and unsupervised learning, where labeled data is scarce.

- They can learn from and generate new examples to improve training effectiveness.

4. Creative Applications (AI Focus)

- Enables natural language generation, image synthesis, music composition, etc.
- Powers tools like ChatGPT, DALL·E, and other generative AI applications.

5. Anomaly Detection

- By Modelling normal data patterns, generative models can identify outliers or anomalies effectively.

6. Simulation and Decision Support

- Used in AI for simulating future scenarios, predicting possible outcomes, or imagining environments.
- Helps in tasks like planning and reinforcement learning.

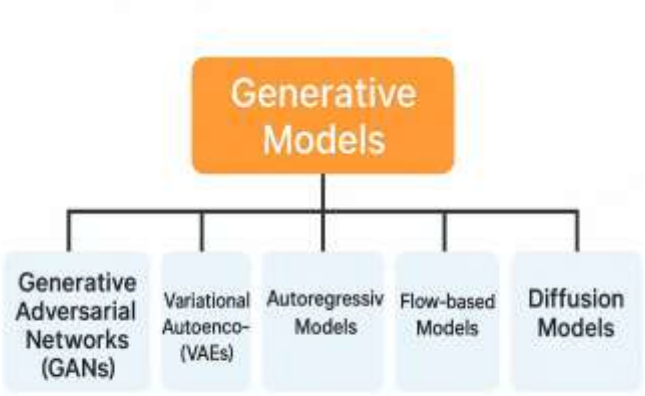
7. Multimodal Learning

- Supports learning across multiple data types—like combining text and images (e.g., caption generation).
- Useful in systems that need to interpret and generate across different media.

8. Foundation for Advanced Models

- Underpins many advanced AI architectures: GANs, VAEs, diffusion models, etc.
- Essential for developing flexible, adaptive, and generative AI systems.

Types of Generative Models



GAN (Generative Adversarial Network)

- Introduced in **2014 by Ian Goodfellow**.

- GANs **do not require labelled data** to train.
- The model learns to generate data by **learning the distribution** of the input data.
- It only needs **samples from the real data** (e.g., images) and **random noise** to train the Generator.
- GANs train two networks (Generator and Discriminator) using **real data without labels**. That's why they are an **unsupervised** branch of **Machine Learning**.

- GANs consist of **two competing neural networks**:

Generator (G):

- Takes random noise as input.
- Tries to **generate fake data** that looks like real data. Learns to **fool the Discriminator**.

Discriminator (D):

- Takes real or fake data as input.
- Learns to **classify data as real or fake**.
- Helps the Generator improve by giving feedback.

How GANs Work

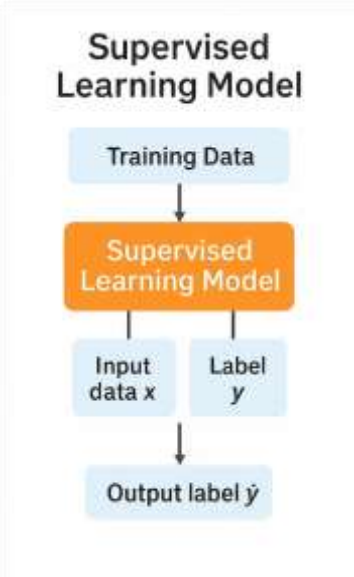
Working Steps of a GAN

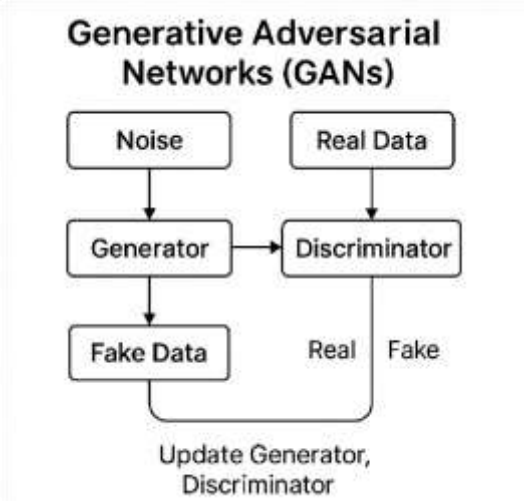
1. **Input Random Noise (z):**This noise is fed into the **Generator**.
2. **Generator Creates Fake Data (G(z)):**The Generator uses the noise to create **fake data samples** (e.g., fake images).
3. **Discriminator Evaluates Real vs. Fake:**
   - The Discriminator receives both:
     - **Real data samples** from the training dataset.
     - **Fake data samples** from the Generator. ○ It tries to **classify them** as real (1) or fake (0).
4. **Loss Calculation (Minimax Loss Function):**
   - The goal is a **zero-sum game**:
     - Discriminator tries to **maximize** the probability of correctly identifying real/fake.
     - Generator tries to **minimize** the probability that the Discriminator detects its fake output.
5. **Update Discriminator Weights:** Use gradient descent to update **Discriminator weights** to better distinguish between real and fake.
6. **Update Generator Weights:** The Generator is trained using feedback from the Discriminator. It updates weights to produce better (more realistic) fake data and **fool the Discriminator**.
7. **Repeat Training Loop:** ○ Steps 1–6 are repeated for many epochs. ○ Over time, the Generator improves so much that the Discriminator **can't tell real from fake** (outputs ~0.5 for both).

*This is like a **forger (Generator)** trying to create fake paintings and a **detective (Discriminator)** trying to catch them.*

Use Cases for GANs

- **Image generation** (e.g., faces, landscapes).
- **Data augmentation** (especially for training small datasets).
- **Synthetic medical data** to train AI without privacy risks.
- **Fashion design**, **art creation**, and **deepfake detection**.



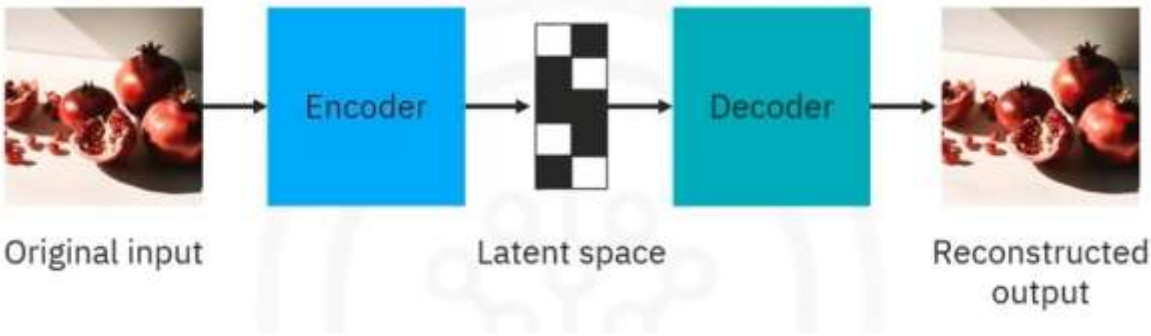


Variational Autoencoder (VAE):

A VAE is a type of neural network that learns to compress data into a structured latent space and can **generate new, realistic data** by sampling from that space.

Key Features

1. **Versatile Data Handling:** training data, making them suitable for a wide range of use.
2. **Dimensionality Reduction:** They reduce the dimensionality of data, which helps in creating improved versions of the original data.
3. **Probabilistic Model:** VAEs utilize probabilistic methods to learn the underlying distribution of the data.



Mechanism

VAEs consist of two main components: an encoder and a decoder.

1. **Encoder:**
   - The encoder is a neural network that processes the input data to identify its most important features.
   - It compresses the data into a lower-dimensional representation, known as the latent space.
   - This involves learning the probability distribution of the data and isolating key variables.
   - The compressed data is stored in the latent space, which serves as a compact representation of the input data.
2. **Latent Space:**
   - The latent space is a mathematical space within the model's architecture where the compressed data is stored.

- It represents the data in a reduced-dimensional format, making it easier to manipulate and generate new samples.

3. **Decoder:**
   - The decoder is another neural network that takes the compressed representation from the latent space and reconstructs it into the original data format.
   - It performs the reverse operation of the encoder, aiming to generate data that closely resembles the original input.
   - The training process involves minimizing the difference between the input data and the reconstructed output using a maximum likelihood principle.

Training Process

- **Maximum Likelihood Principle:** VAEs are trained to minimize the difference between the original input and the reconstructed output, ensuring the generated data is as close to the original as possible.
- **Continuous Latent Space:** Despite being trained in a static environment, the latent space in VAEs is continuous, allowing for the generation of new data samples by randomly sampling from the learned distribution.

Applications

1. Image Synthesis
2. Data Compression
3. Anomaly Detection
4. Industry Examples:
   - Entertainment
   - Finance Health care

Autoregressive Models

An **Autoregressive (AR) Model** is a generative model that **predicts the next element** in a sequence **based on previous elements**. It builds sequences **step by step**, where each prediction depends on the outputs generated so far.

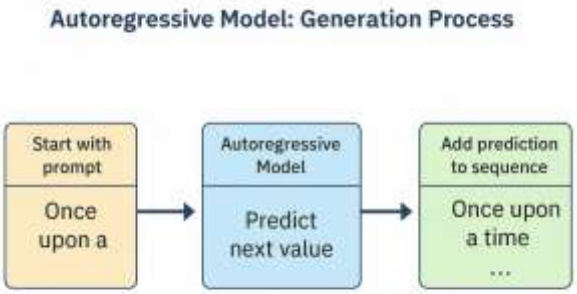
Structure of an AR

Step-by-step flow:

1. **Start with a prompt** (e.g., first word or image pixel).
2. **The model predicts the next value** using previous values. (RNN, Transformer, etc.,)
3. **Add the prediction to the sequence**.
4. Repeat the process to build the full output.

Autoregressive Equation:

$$P(x) = P(x_1) \cdot P(x_2|x_1) \cdot P(x_3|x_1, x_2) \cdot \ldots \cdot P(x_n|x_1, ..., x_{n-1})$$



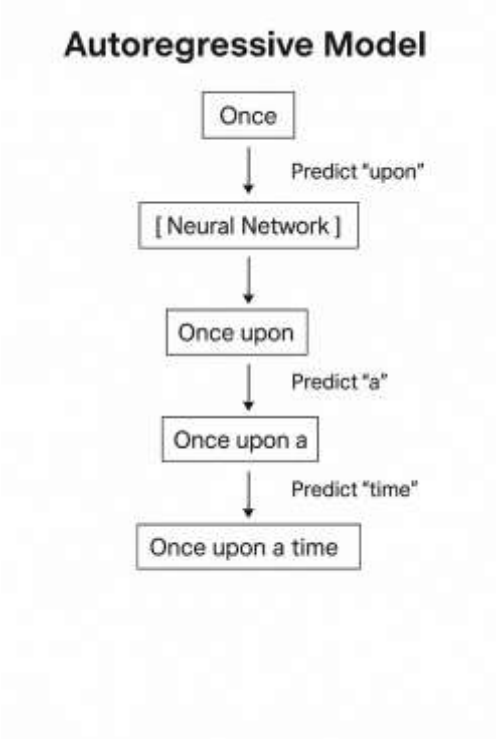
Example: Word-by-Word Storytelling

1. **Prompt**: You say "Today,"
2. **Model predicts**: "I"

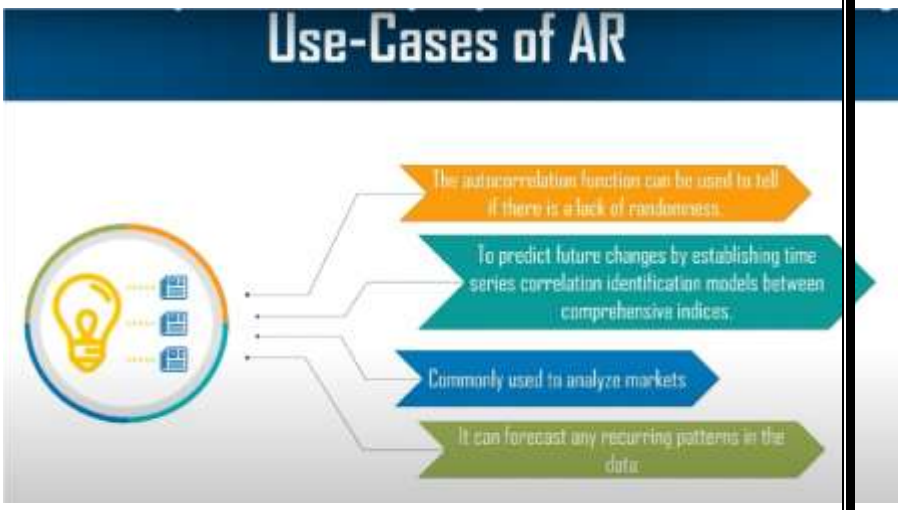
Because "Today I" is a common phrase based on its training.

3. The sentence so far: "Today I"
4. **Model predicts**: "went" ○ Now your sentence is "Today I went"
5. **Model predicts next**: "to"
6. Then: "the"
7. Then: "park."

Resulting in: **"Today I went to the park."**



Domain	Example Models	What They Generate
Text	GPT, LLaMA	Sentences, code, answers
Audio	WaveNet	Speech, music
Images	PixelRNN, PixelCNN	Pixel-wise image generation
Music	Music Transformer	Notes, compositions



Vector quantized Diffusion models

What is a Diffusion Model?

A **Diffusion Model** is a type of **generative model** that learns to **generate new data** (like images, text, or audio) by **reversing a process that adds noise** to data.

Think of it like learning to "paint an image from pure noise" — step by step, clearer and clearer.

1. **Forward Diffusion Process**
2. **Reverse Diffusion Process**
3. **Conditional Diffusion**

Vector Quantization (VQ)

Convert continuous data (e.g., image pixels) into **discrete tokens** from a learned codebook. Used in models like **VQ-VAE (Vector Quantized Variational Autoencoder)**.

Input → Encoder → Discrete tokens (via nearest codebook vector) → Decoder → Reconstructed input

Vector Quantized Diffusion Model

VQ-Diffusion = **Diffusion process in a discrete latent space** (of VQ tokens), not in pixel space.

1. **Train a VQ model** (e.g., VQ-VAE) to compress data into discrete latent tokens.
2. **Use a diffusion model** on these discrete tokens instead of raw data:
3. Apply **noise** by randomly replacing tokens with others.
4. Train model to **reverse** this noising process (denoise discrete tokens).
5. **Reconstruct the original data** using the VQ decoder.

Image → [VQ Encoder] → Tokens → [Discrete Diffusion Model] → Denoised Tokens → [VQ Decoder] → Image **1.**

Forward Diffusion Process

The **forward process** gradually adds noise to the data. After many steps, the data becomes pure Gaussian noise.

2. Reverse Diffusion Process

The **reverse process** is learned by a neural network to gradually remove noise from a random noisy sample, and generate new data (like an image, audio, etc.).

3. Conditional Diffusion

In **conditional diffusion**, we guide the reverse process using **extra information** like text prompts, class labels, or images.

For example, in **text-to-image generation**, the condition is a **text prompt** like “a cat wearing sunglasses”.

Text Prompt: "A cat wearing sunglasses"

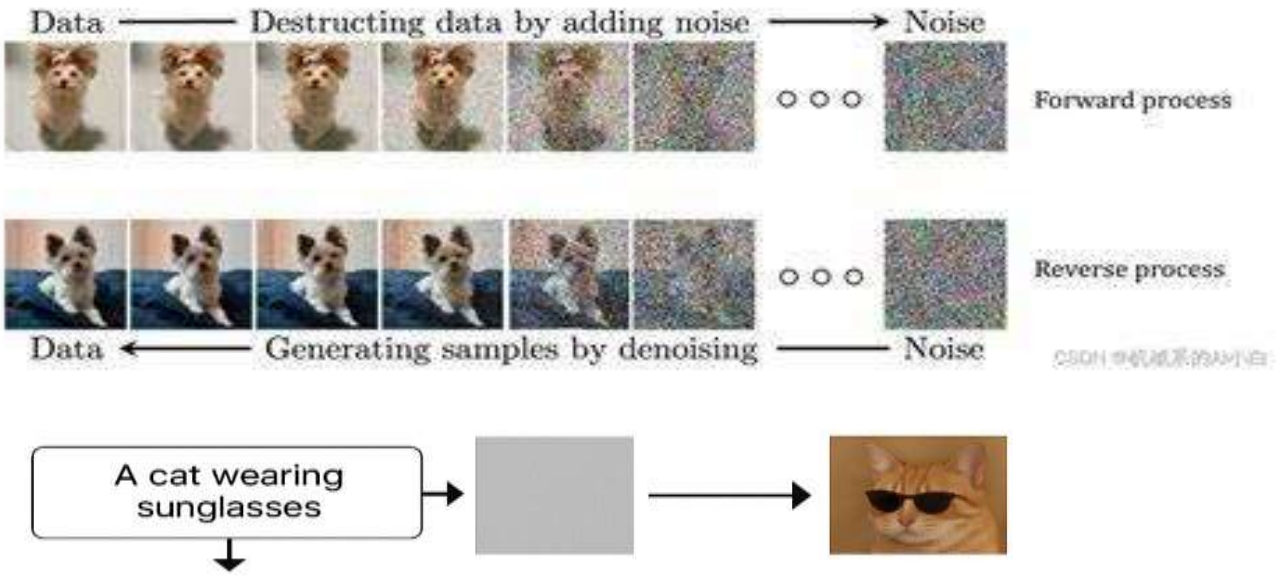
↓

Conditioning input

↓

Noise → Model(x_t | text) → Denoised Step

→ ... repeat ... → Final Image 🐱



Understanding Probabilistic Modelling and the Generative Process

Probabilistic Modelling is a **mathematical approach** to represent uncertainty and variability in the world using **probability distributions**.

It tells us how likely different outcomes are, given our observations.

Imagine you're predicting tomorrow’s weather.

- You don't say: "It **will** rain."
- You say: "There's a **70% chance** of rain."

That's **probabilistic reasoning**.

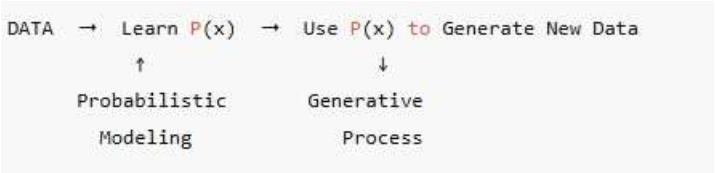
Generative process is a way of **creating data** using a probabilistic model.

You sample from a **known distribution**, often in steps, to generate realistic examples.

"Now that I understand how 3s look, let me **generate** one!"

Why is This Useful?

- Uncertainty-aware predictions
- Generates realistic data (text, images, speech)
- Core of **Generative AI**, including **ChatGPT**, **DALL·E**, **Stable Diffusion**



Model	Type	Description
VAE	Probabilistic	Learns latent space + reconstructs data
Diffusion	Probabilistic	Adds/removes noise using distributions
Autoregressive (e.g., GPT)	Probabilistic	Predicts the next token using conditional probabilities
GANs	Not fully probabilistic	Adversarial training, less explicit probability

Challenges of Generative Modelling

Generative models aim to learn the underlying data distribution and generate new samples, but this is not easy.

1. Mode Collapse (in GANs)

- The model generates only a few types of samples repeatedly.
- Ignores the full diversity of the data.

Example: GAN trained on digits may only generate "3" and "7", never "2" or "9".

Fixes: Wasserstein GAN, better training strategies, multiple discriminators.

2. Training Instability

- Loss functions (especially in GANs) may not converge or behave unpredictably.
- Models oscillate or diverge.

Fix: Careful model tuning, normalization, and gradient clipping.

3. High Computational Cost

- Generative models (especially diffusion models) require:
  - Many forward/backward passes
  - High-dimensional data (images, videos)
  - Long training times

Example: Diffusion models like Stable Diffusion need hundreds of denoising steps to generate one image.

4. Likelihood Estimation is Hard

- Accurately computing or approximating the likelihood of data under the model is difficult in high dimensions.

This affects models like VAEs and Normalizing Flows, which rely on tractable probability functions.

5. Evaluation is Tricky

There is no single metric that perfectly captures the quality and diversity of generated samples.

Metric	Problem
Inception Score (IS)	Biased toward certain classes
FID (Fréchet Inception Distance)	May not align with human judgment
Human evaluation	Time-consuming, subjective

6. Overfitting / Memorization

- Model may memorize training data rather than learning to generalize.
- Especially dangerous in sensitive domains (e.g., healthcare, faces).

E.g., Generated images might be nearly identical to training samples — privacy risk.

7. Controlling Output is Hard

- It's difficult to control what the model generates:
  - Want a cat? It makes a dog.
  - Want a smiling person? It ignores emotion.

Conditional models like text-to-image or class-conditioned GANs are better, but still imperfect.

8. Bias and Ethical Issues

- If training data is biased, the model amplifies that bias.
- Can reinforce harmful stereotypes or misinformation.

Needs:

- Better datasets
- Fairness-aware training

9. Safety and Misuse

- Generative models can be used to:
  - Create fake content (deepfakes) ○ Imitate voices, faces, and documents This raises ethical and legal concerns.

10. Multimodal Learning Challenges

- Combining text + image + audio is hard:
  - Different modalities have different structures and noise levels. ○ Aligning them correctly is non-trivial (e.g., text matching correct image region).

Future of Gen AI

1. Multimodal Generative AI(Text, Images, Audio, Video) Examples:

- Generate a video from a text prompt ("A tiger walking in the snow")

Already emerging: OpenAI's GPT-4o, Google Gemini, Meta's I-JEPA

2. Smarter, Context-Aware Generation

- Gen AI will understand user context, intent, and history Tailored generation:
  - Personalized learning material ○ Health advice using personal history
  - Business reports are auto-generated for your organization

Future: Models with long-term memory + reasoning (e.g., ChatGPT with memory)

3. Better Control, Customization, and Alignment

Users will steer AI generation more easily using: Structured prompts, Natural conversation, Visual editing tools

Control mechanisms: Prompt tuning, Retrieval-augmented generation (RAG), Fine-tuned personal assistants

4. Ethical, Fair, and Safe Generative AI

Future Gen AI will be built with: Bias detection, Fairness constraints, Consent-aware data usage, Digital watermarking (to detect AI-generated content)

Expected: Government regulations for watermarking, data transparency, and copyright.

5. Industry-Specific Gen AI Applications

Industry	Future Applications

Healthcare	Drug discovery, medical report generation
Education	Personalized tutoring, auto-grading, and simulations
Business	Report automation, decision support
Art & Design	AI co-creators for media, games, and fashion
Law	Contract drafting, legal research

6. Open-Source and Democratization

- More open-source models (e.g., Mistral, Stable Diffusion, LLaMA)
- On-device AI (e.g., on mobile phones and edge devices)
- Low-code/no-code Gen AI platforms

This will make custom AI tools accessible to students, small businesses, and creators.

7. Scaling + Efficiency

- Smaller, faster, energy-efficient models
- Compression, quantization, and distillation techniques
- Edge AI: Run Gen AI on laptops, phones, even microcontrollers

8. Global & Inclusive Gen AI

- Support for more languages, dialects, and cultures
- Localized content generation
- Indigenous data & storytelling models

9. Regulation and Responsible DevelopmentGovernments and organizations will shape:

- Copyright laws
- AI labeling & authentication
- Transparency & accountability

Users will demand:

- "What was your training data?"
- "Can I remove my data from the model?"

10. Beyond Generation: Reasoning, Planning & Interaction

The boundary between generative AI and AGI (Artificial General Intelligence) is narrowing.

- Models will not just generate, but also:
  - Plan tasks
  - Debate topics
  - Simulate environments
  - Interact with tools (code, APIs)

Ethical Aspects of AI

Ethics in AI deals with ensuring that AI technologies are **fair**, **safe**, **transparent**, and **respectful of human rights**.

Key Ethical Issues:

Ethical Concern	Description
Bias & Fairness	AI may reinforce existing biases from data (e.g., gender or racial bias).
Privacy	Use of personal data raises privacy risks (e.g., facial recognition).
Accountability	Who is responsible when AI systems cause harm or error?
Transparency	Black-box models lack explainability. Users need to understand decisions.
Autonomy	AI should not undermine human decision-making or freedom.
Safety & Security	Malfunctioning or adversarial attacks on AI can cause harm.
Job Displacement	AI can automate jobs, affecting employment and social stability.

Responsible AI

Responsible AI ensures that AI is developed and used in a way that aligns with ethical principles and societal values.

Core Principles of Responsible AI:

Principle	Description
Fairness	Avoid discrimination and ensure equitable outcomes.
Transparency	Clearly explain how AI models work and make decisions.
Accountability	Assign responsibility to developers and organizations.
Privacy Protection	Safeguard user data and give control to individuals.
Robustness	Ensure reliability, accuracy, and resistance to manipulation.
Human-Centered	Design AI to enhance, not replace, human roles.
Inclusiveness	Engage stakeholders from diverse backgrounds in AI design.

Use Cases of Responsible AI

Sector	Use Case	Description
Healthcare	AI Diagnostics	AI helps detect diseases (e.g., cancer, diabetic retinopathy) ethically.
Finance	Fraud Detection	AI identifies suspicious transactions while protecting user privacy.
Education	Adaptive Learning Systems	Personalized learning while ensuring data protection and inclusion.
Agriculture	Smart Farming	AI predicts crop yield, optimizing resources without harming communities.
Government	Public Policy Analysis	AI supports data-driven decisions transparently and fairly.

Environment	Climate Modeling	AI forecasts environmental changes and promotes sustainability.
Law & Justice	Predictive Policing	Used cautiously to avoid racial profiling or unfair targeting.

Best Practices for Implementing Ethical AI:

- Conduct **AI impact assessments**.
- Include **interdisciplinary teams** (ethics, law, tech).
- Use **auditable** and **interpretable models**.
- Apply **data minimization** and **differential privacy** techniques. Train models on **diverse, representative datasets**.

Suggested Tools & Frameworks:

- AI Fairness 360 (IBM)
- Microsoft Responsible AI Standard
- Google's AI Principles
- EU's AI Act and Ethics Guidelines
- IEEE Ethically Aligned Design

UNIT -2

Generative Models for Text: Language Models Basics, building blocks of Language models, Transformer Architecture, Encoder and Decoder, Attention mechanisms, Generation of Text, Models .like BERT and GPT models, Generation of Text, Auto encoding, Regression Models, Exploring Chat GPT, Prompt Engineering: Designing Prompts, Revising Prompts using Reinforcement Learning from Human Feedback (RLHF), Retrieval Augmented Generation, Multimodal LLM, Issues of LLM like hallucination.

Introduction to Generative Models for Text: (Language Models Basics)

Generative Models for Text are a core component of **Generative Artificial Intelligence**, designed to automatically create meaningful, coherent, and human-like text. These models learn the patterns, structure, grammar, and semantics of language from large volumes of text data and then use this learned knowledge to generate new text based on a given prompt or context.

At a high level, generative text models work by predicting the most probable next word or token in a sequence, given the previous words. By repeating this process iteratively, they can produce complete sentences, paragraphs, or even long documents. Early approaches relied on statistical language models, while modern systems primarily use **deep learning**, especially **neural networks**.

The most influential generative models for text today are based on the **Transformer architecture**, such as **GPT (Generative Pre-trained Transformer)**, **BERT (for understanding, not generation)**, and **T5**. These models are typically trained in two stages: **pre-training**, where they learn general language patterns from massive datasets, and **fine-tuning**, where they adapt to specific tasks like summarization, translation, question answering, or dialogue.Generative text models are widely used in applications such as chatbots, virtual assistants, content generation, code generation, automatic summarization, and sentiment-aware text creation. Despite their powerful capabilities, they also face challenges like bias, hallucination, and ethical concerns, which are active areas of research in Generative AI. Overall, generative models for text represent a major advancement in AI, enabling machines to understand and produce natural language in a way that closely resembles human communication.A **Language Model (LM)** is a fundamental building block of **Generative Artificial Intelligence**, especially for **text generation tasks**. A language model is designed to **understand, learn, and generate natural language** by estimating the probability distribution of word sequences. In Generative AI, language models enable machines to produce coherent sentences, paragraphs, and documents that resemble human-written text.

The primary objective of a language model is to **predict the next word (or token)** in a sequence based on the previously observed words. By repeatedly applying this prediction process, the model can generate complete and meaningful text outputs.

A **language model** is a probabilistic model that assigns a probability to a sequence of words W=w1,w2,...,wn

W = w_1, w_2, ..., w_n

W=w1,w2,...,wn as:

$P(W)=P(w1)\cdot P(w2|w1)\cdot P(w3|w1,w2)\cdots P(wn|w1,...,wn-1)$

This probabilistic formulation allows the model to capture the **syntactic and semantic structure** of language

Language models are the **core engine** behind generative text systems. They help in:

- Learning grammar, sentence structure, and word usage
- Understanding context and meaning from large text corpora
- Generating fluent and contextually relevant text
- Supporting advanced applications like chatbots, summarization, translation, and content creation

Without language models, text generation in Generative AI would not be possible.

Types of Language Models

1. Statistical Language Models

Statistical language models are the earliest form of language modeling. They rely on **probability and frequency counts** of word sequences.

- **Unigram Model**: Considers each word independently
- **Bigram Model**: Predicts a word based on one previous word
- **Trigram Model**: Uses two previous words for prediction

These models are easy to implement but suffer from **data sparsity**, **limited context**, and poor handling of long sentences.

2. Neural Language Models

Neural language models use **neural networks** to overcome the limitations of statistical approaches.

- They represent words using **word embeddings**
- Capture semantic similarity between words
- Learn complex patterns in text

Common neural architectures include:

- **Feedforward Neural Networks**
- **Recurrent Neural Networks (RNNs)**
- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Units (GRUs)**

These models perform better than n-grams but struggle with **long-range dependencies** and sequential processing inefficiencies.

3. Transformer-Based Language Models

Modern Generative AI systems rely on **Transformer-based language models**.

- Use **self-attention mechanisms** to capture relationships between all words in a sentence
- Process text in parallel, improving efficiency
- Handle long-context dependencies effectively

Examples include:

- **GPT (Generative Pre-trained Transformer)** – text generation
- **BERT** – language understanding
- **T5** – text-to-text framework

Transformers represent the state-of-the-art in generative text modeling. And Language models are typically trained in two stages:

1. **Pre-training**
    - Trained on massive unlabeled text data
    - Learn general language patterns and world knowledge
    - Objective: next-token prediction or masked word prediction
2. **Fine-tuning**
    - Adapted for specific tasks like chatbots, summarization, or QA
    - Uses task-specific labeled data

This training strategy enables models to generalize well across tasks.

Tokenization in Language Models

Instead of processing raw words, modern language models operate on **tokens**.

- Tokens can be words, sub-words, or characters
- Popular tokenization techniques include **BPE**, **WordPiece**, and **SentencePiece**
- Tokenization helps manage vocabulary size and handle unknown words

Language models power several Generative AI applications, such as:

- Text generation and story writing
- Conversational agents and chatbots
- Machine translation
- Text summarization
- Code generation
- Question answering systems

These applications demonstrate the versatility of language models.

BUILDING BLOCKS OF LANGUAGE MODELS

Building Blocks of Language Models in Generative AI

Language models in Generative AI are systems designed to understand, generate, and manipulate human language. They are built using several fundamental components that work together to process text data and produce meaningful outputs. The important building blocks are explained below.

1. Text Data (Training Corpus)

The first and most important building block of a language model is large-scale text data.
Language models are trained on massive datasets collected from books, articles, websites, research papers, and conversational text. This data helps the model learn grammar, vocabulary, sentence structure, facts, and contextual relationships between words.
High-quality, diverse, and well-preprocessed data improves the accuracy, fluency, and generalization ability of the model.

2. Tokenization

Tokenization is the process of breaking text into smaller units called tokens.
Tokens can be characters, subwords, or whole words depending on the tokenizer used. Modern language models often use subword tokenization techniques like Byte Pair Encoding (BPE) or WordPiece to efficiently handle rare words and reduce vocabulary size.
Tokenization converts raw text into a numerical form that the model can understand.

3. Embeddings

Embeddings are dense numerical vector representations of tokens.
Each token is mapped into a high-dimensional vector space where semantic meaning is captured. Tokens with similar meanings tend to have similar embeddings.
Embeddings allow the model to understand relationships such as similarity, analogy, and context between words, forming the foundation for semantic understanding.

4. Model Architecture (Transformer)

The core building block of modern language models is the Transformer architecture.
Transformers use layers of neural networks that process tokens in parallel, making them efficient and scalable.
Key components inside the Transformer include:

- Multi-layer neural networks
- Attention mechanisms
- Feed-forward networks

This architecture enables the model to capture long-range dependencies in text effectively.

5. Attention Mechanism

Attention is a crucial building block that allows the model to focus on relevant words in a sentence while generating or understanding text.
Using self-attention, each token attends to other tokens in the sequence and learns their importance.
This helps the model understand context, word relationships, and sentence meaning more accurately than traditional sequential models.

6. Positional Encoding

Since transformers process tokens in parallel, they need a way to understand word order.
Positional encoding provides information about the position of each token in a sentence.
By adding positional information to embeddings, the model can distinguish between sentences like “Dog bites man” and “Man bites dog”.

7. Training Objective and Loss Function

Language models are trained using objectives such as next-word prediction or masked language modeling.
A loss function (commonly cross-entropy loss) measures the difference between the predicted output and the actual word.
Through backpropagation and optimization techniques like gradient descent, the model gradually improves its predictions.

8. Parameters and Weights

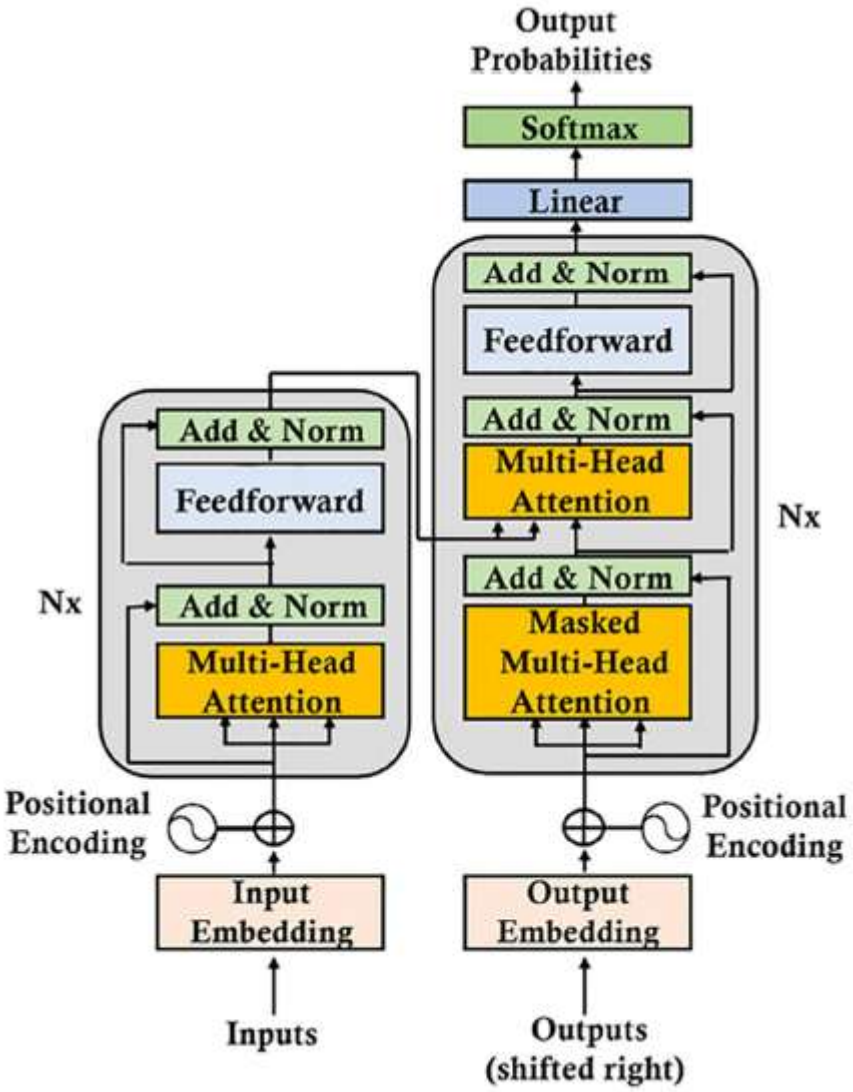
Parameters are the learned weights of the neural network.
Modern generative language models contain millions or even billions of parameters.
These parameters store linguistic knowledge learned during training and directly influence the model’s ability to generate fluent and context-aware text.

9. Decoding and Text Generation

During inference, decoding strategies are used to generate text.
Common methods include greedy decoding, beam search, top-k sampling, and nucleus (top-p) sampling.
These techniques control randomness, coherence, and creativity in generated text.

10. Fine-Tuning and Alignment

After pre-training, models are often fine-tuned on task-specific data or aligned using human feedback.
This helps the model follow instructions, reduce harmful outputs, and improve usefulness in real-world applications.Transformer Architecture



The **Transformer architecture** is a core model used in **Generative AI** for tasks such as text generation, translation, and language modeling. It is based entirely on the **attention mechanism**, avoiding recurrence and enabling parallel processing.

- **Encoder and Decoder**

The **Transformer** is a deep learning architecture introduced to handle **sequence-to-sequence tasks** such as machine translation, text summarization, question answering, and text generation. Unlike RNNs and LSTMs, the Transformer does **not use recurrence**. Instead, it relies completely on **attention mechanisms**, enabling parallel processing and better handling of long-range dependencies.

The Transformer mainly consists of two blocks:

1. **Encoder**
2. **Decoder**

1. Encoder in Transformer

The **encoder** is responsible for **understanding the input sequence** and converting it into a rich contextual representation.

Structure of Encoder

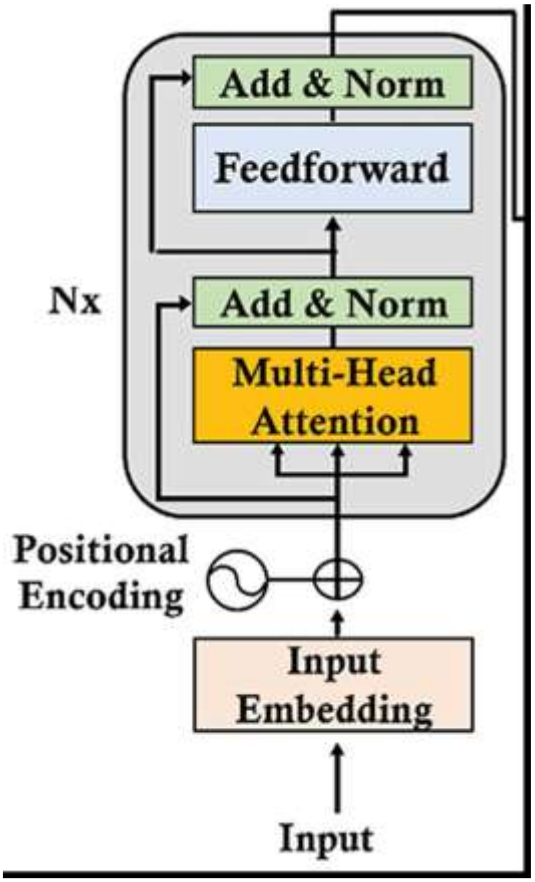
The encoder consists of N identical layers, each containing multi-head self-attention and feed-forward networks with residual connections and layer normalization to generate contextual representations of input tokens.

- The encoder is made up of **N identical layers** (usually 6).
- Each encoder layer has **two main sub-layers**:
    1. Multi-Head Self-Attention
    2. Feed-Forward Neural Network

Working of Encoder

1. **Input Embedding**
    - The input text is first converted into vectors using **word embeddings**.
    - These embeddings represent semantic meaning of words.
2. **Positional Encoding**
    - Since the Transformer has no recurrence, **positional encoding** is added to embeddings to provide information about word order in the sequence.
3. **Multi-Head Self-Attention**
    - Each word in the input attends to **all other words** in the sentence.
    - This helps the model understand relationships such as subject–verb, context, and dependencies.
    - Multiple attention heads allow learning different types of relationships simultaneously.
4. **Add & Normalize**
    - Residual connections are added to prevent vanishing gradients.
    - Layer normalization stabilizes training.
5. **Feed-Forward Neural Network (FFN)**
    - A fully connected neural network applied independently to each position.
    - Introduces non-linearity and improves representation power.
6. **Output of Encoder**

- The final output is a set of **context-aware representations** passed to the decoder.



2. Decoder in Transformer

The **decoder** is responsible for **generating the output sequence** (text) one token at a time.

Structure of Decoder

- Generates fluent and context-aware text
- Maintains grammatical and semantic coherence
- Enables autoregressive text generation
- Forms the core of **decoder-only models** such as GPT (without encoder)

- The decoder also consists of **N identical layers**.
- Each decoder layer has **three sub-layers**:
    1. Masked Multi-Head Self-Attention
    2. Encoder–Decoder (Cross) Attention
    3. Feed-Forward Neural Network

Working of Decoder

1. **Output Embedding and Positional Encoding**
    - The previously generated output tokens are converted into embeddings.
    - Positional encoding is added to preserve order.
2. **Masked Multi-Head Self-Attention**
    - Masking ensures that the decoder **cannot see future tokens**.

- This maintains the autoregressive nature required for text generation.

3. **Encoder–Decoder Attention (Cross Attention)**
   - The decoder attends to the **encoder output**.
   - This helps align generated words with relevant input context.
4. **Add & Normalize**
   - Residual connections and normalization are applied after each sub-layer.
5. **Feed-Forward Neural Network**
   - Further processes the combined information.
6. **Linear Layer and Softmax**
   - Converts decoder outputs into probabilities over the vocabulary.
   - The word with highest probability is selected as output.

Attention Mechanisms

The **attention mechanism** is the core idea behind the Transformer architecture. It allows the model to **focus on the most relevant parts of the input sequence while processing or generating data**, instead of treating all words equally. In Generative AI, attention helps models understand **context, relationships, and dependencies** between words, even when they are far apart in a sentence. In traditional models like RNNs and LSTMs, information is processed **sequentially**, which makes it difficult to capture **long-range dependencies** and slows down training. The attention mechanism overcomes these problems by:

- Processing all tokens **in parallel**
- Capturing **global context**
- Giving **different importance** to different words
- Improving both **speed and accuracy**

Types of Attention in Transformers

1. Self-Attention

- Each word attends to **all other words in the same sentence**
- Helps the model understand **context within a sentence**
- Used in both **encoder and decoder**

Example:
In the sentence *"The boy who was hungry ate food"*, self-attention helps link **"boy"** with **"ate"**.

2. Multi-Head Attention: Multiple Perspectives

Instead of having just one "attention" calculation, Transformers use **Multi-Head Attention**. This means the model runs several self-attention processes in parallel.

- One "head" might focus on **grammar** (e.g., matching a subject to a verb).
- Another head might focus on **vocabulary** (e.g., finding synonyms).
- A third might look at **long-range dependencies** (e.g., connecting a pronoun at the end of a paragraph to a name at the start)

3. Encoder–Decoder Attention (Cross-Attention)

- Used only in the **decoder**

- Decoder attends to **encoder outputs**
- Helps in tasks like **translation and summarization**

4. Masked Attention (Causal Attention)

- Used in **generative models**
- Prevents the model from seeing **future tokens**
- Ensures left-to-right generation

This is crucial for **text generation**, where the next word must depend only on previous words.

Generation of Text

Text generation is the process of using artificial intelligence (AI) to produce human-like written content. By leveraging advanced machine learning techniques, particularly Natural Language Processing (NLP), AI models can generate coherent and contextually relevant text. This technology is transforming how we create content, making writing faster and more efficient.

How Text Generation Works?
Text generation works by training AI models on vast datasets of text. These models analyse thousands, if not millions, of documents to recognize patterns, sentence structures, and word relationships. Just as a student learns a language by reading books and practicing conversations, AI models learn from text data to generate coherent and meaningful responses.

When given a prompt, the model predicts the most likely next word or phrase based on what it has learned. This process is similar to how predictive text on your phone suggests words while typing, but on a much more advanced level. By continuously refining predictions and incorporating context, AI models can generate entire paragraphs or even full articles with logical flow and coherence.

Different Approaches to Text Generation
Text generation models rely on different techniques to generate text:

- **Autoregressive models**: These models, such as GPT-4, generate text by predicting one word at a time based on the sequence of words that came before it. This approach ensures that the generated text follows a logical and coherent flow, much like how humans write by thinking about the next word based on previous context.

- **Seq2Seq models:** These models are commonly used in tasks like machine translation, where an input sequence (such as a sentence in one language) is transformed into an output sequence (the translated sentence in another language). This method is effective for applications where structured input must be mapped to structured output, ensuring meaningful conversions.

- **Fine-tuned models:** Pre-trained AI models can be further customized using specific datasets to specialize in particular domains, such as generating medical reports, legal documents, or financial summaries. By fine-tuning these models with domain-specific data, they can generate more accurate and contextually relevant outputs tailored to specialized fields.Some of the most powerful models for text generation include:

- **GPT (Generative Pre-trained Transformer)** by OpenAI
- **PaLM 2** by Google
- **Claude** by Anthropic
- **LLaMA** by Meta AI

Real-World Applications of Text Generation
Text generation has numerous applications across different industries:

1. Content Creation
AI-driven tools like ChatGPT and Jasper AI assist in generating blog posts, product descriptions, social media posts, and more. These tools can significantly reduce content production time while maintaining quality.For example, if a company needs to create hundreds of product descriptions for an online store, AI can generate them quickly, ensuring consistency and clarity.

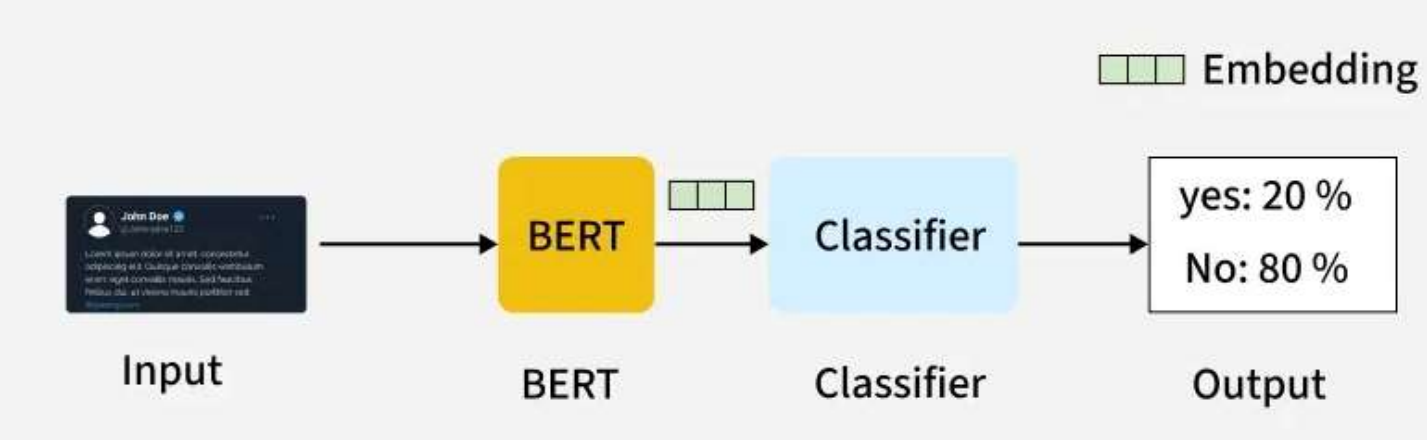
2. Chatbots & Virtual Assistants
AI-powered chatbots and virtual assistants like Siri, Alexa, and Google Assistant use text generation to provide real-time, conversational responses to user queries. If you've ever asked your phone's assistant about the weather or to set a reminder, you've used text generation in action.

3. Language Translation
Services like Google Translate leverage AI to generate text in different languages, improving cross-lingual communication. Imagine traveling to a foreign country and using an app to instantly translate a menu—that's text generation making life easier.

4. Summarization
Text summarization tools use AI to condense long documents, articles, or research papers into concise summaries while retaining the key information. For students, this can be useful when preparing for exams, as AI can summarize large textbooks into key points.

5. Code Generation
Developers use AI tools like GitHub Copilot to generate and complete code snippets, accelerating software development. Think of it as a smart assistant for programmers, helping them write efficient code faster.

BERT Model



BERT (Bidirectional Encoder Representations from Transformers) leverages a transformer-based neural network to understand and generate human-like language. BERT employs an encoder-only architecture. In the original Transformer architecture, there are both encoder and decoder modules. The decision to use an encoder-only architecture in BERT suggests a primary emphasis on understanding input sequences rather than generating output sequences.
Traditional language models process text sequentially, either from left to right or right to left. This method limits the model's awareness to the immediate context preceding the target word. BERT uses a bi-directional approach considering both the left and right context of words in a sentence, instead of analyzing the text sequentially, BERT looks at all the words in a sentence simultaneously.

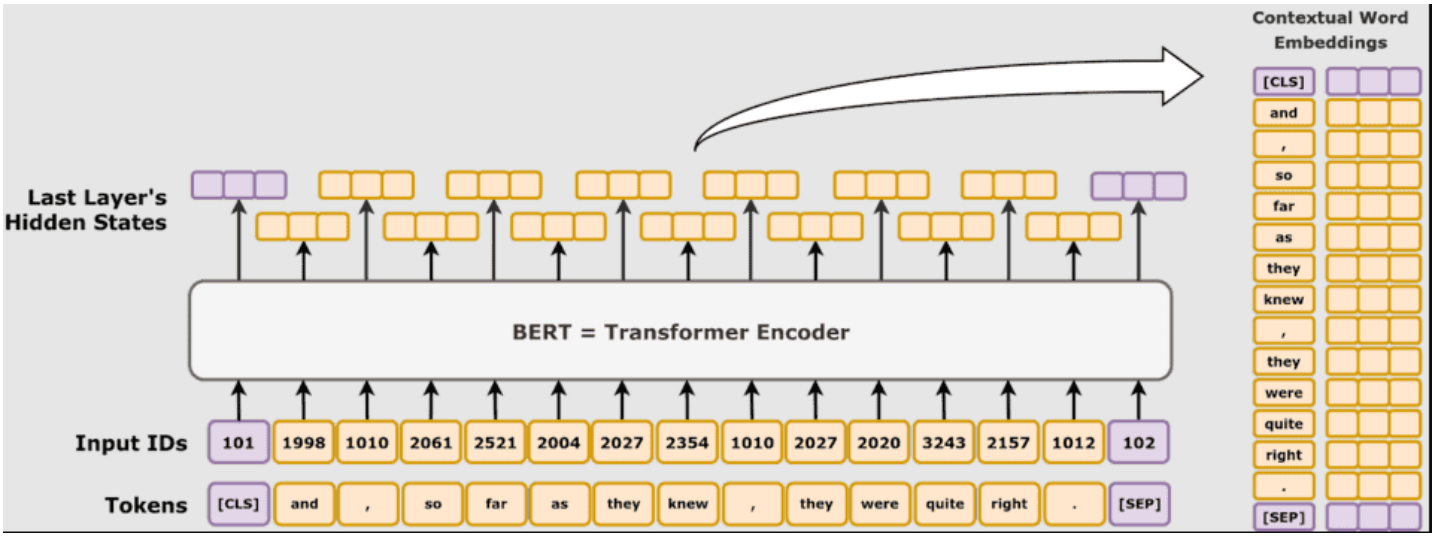
Pre-training BERT Model

The BERT model undergoes Pre-training on Large amounts of unlabeled text to learn contextual embeddings.

- BERT is pre-trained on large amount of unlabeled text data. The model learns contextual embeddings, which are the representations of words that take into account their surrounding context in a sentence.
- BERT engages in various unsupervised pre-training tasks. For instance, it might learn to predict missing words in a sentence (Masked Language Model or MLM task), understand the relationship between two sentences, or predict the next sentence in a pair.

Workflow of BERT

BERT is designed to generate a language model so, only the encoder mechanism is used. Sequence of tokens are fed to the Transformer encoder. These tokens are first embedded into vectors and then processed in the neural network. The output is a sequence of vectors, each corresponding to an input token, providing contextualized representations. When training language models, defining a prediction goal is a challenge. Many models predict the next word in a sequence, which is a directional approach and may limit context learning.



BERT addresses this challenge with two innovative training strategies:

1. Masked Language Model (MLM)
2. Next Sentence Prediction (NSP)

1. Masked Language Model (MLM)

In BERT's pre-training process, a portion of words in each input sequence is masked and the model is trained to predict the original values of these masked words based on the context provided by the surrounding words.

1. BERT adds a classification layer on top of the output from the encoder. This layer is important for predicting the masked words.
2. The output vectors from the classification layer are multiplied by the embedding matrix, transforming them into the vocabulary dimension. This step helps align the predicted representations with the vocabulary space.

3. The probability of each word in the vocabulary is calculated using the SoftMax activation function. This step generates a probability distribution over the entire vocabulary for each masked position.
4. The loss function used during training considers only the prediction of the masked values. The model is penalized for the deviation between its predictions and the actual values of the masked words.
5. The model converges slower than directional models because during training, BERT is only concerned with predicting the masked values, ignoring the prediction of the non-masked words. The increased context awareness achieved through this strategy compensates for the slower convergence.

2. Next Sentence Prediction (NSP)

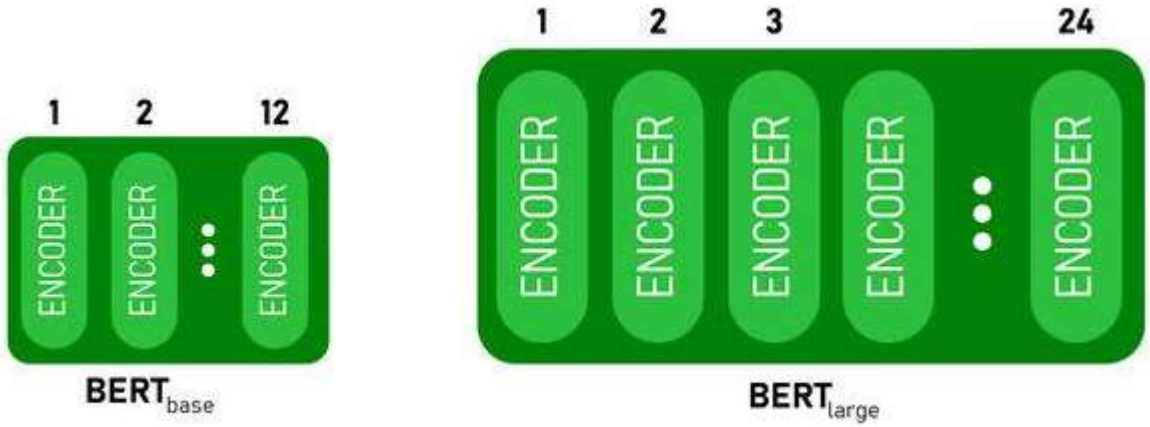
BERT predicts if the second sentence is connected to the first. This is done by transforming the output of the [CLS] token into a 2×1 shaped vector using a classification layer, and then calculating the probability of whether the second sentence follows the first using SoftMax.

1. In the training process, BERT learns to understand the relationship between pairs of sentences, predicting if the second sentence follows the first in the original document.
2. 50% of the input pairs have the second sentence as the subsequent sentence in the original document, and the other 50% have a randomly chosen sentence.
3. To help the model distinguish between connected and disconnected sentence pairs. The input is processed before entering the model.
4. BERT predicts if the second sentence is connected to the first. This is done by transforming the output of the [CLS] token into a 2×1 shaped vector using a classification layer, and then calculating the probability of whether the second sentence follows the first using SoftMax.

BERT Architecture

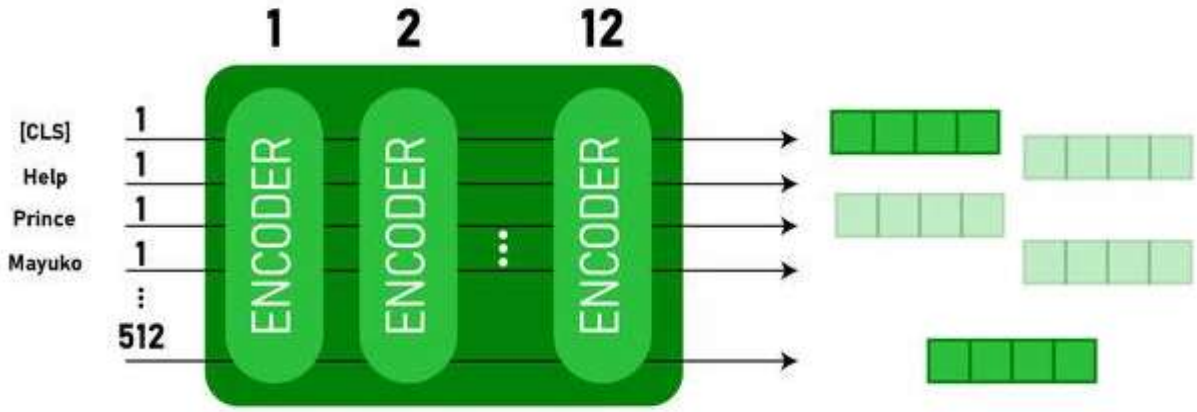
The architecture of BERT is a multilayer bidirectional transformer encoder which is quite similar to the transformer model. A transformer architecture is an encoder-decoder network that uses self-attention on the encoder side and attention on the decoder side.

1. $BERT_{BASE}$ has 12 layers in the Encoder stack while $BERT_{LARGE}$ has 24 layers in the Encoder stack. These are more than the Transformer architecture described in the original paper (6 encoder layers).
2. BERT architectures (BASE and LARGE) also have larger feedforward networks (768 and 1024 hidden units respectively), and more attention heads (12 and 16 respectively) than the Transformer architecture suggested in the original paper. It contains 512 hidden units and 8 attention heads.
3. $BERT_{BASE}$ contains 110M parameters while $BERT_{LARGE}$ has 340M parameters.



BERT BASE and BERT LARGE architecture

This model takes the CLS token as input first, then it is followed by a sequence of words as input. Here CLS is a classification token. It then passes the input to the above layers. Each layer applies self-attention and passes the result through a feedforward network after then it hands off to the next encoder. The model outputs a vector of hidden size (768 for BERT BASE). If we want to output a classifier from this model we can take the output corresponding to the CLS token.



BERT output as Embeddings

Now, this trained vector can be used to perform a number of tasks such as classification, translation, etc. For Example, the paper achieves great results just by using a single layer Neural Network on the BERT model in the classification task.

Generative Pre-trained Transformer (GPT) is a large language model that can understand and produce human-like text. It works by learning patterns, meanings and relationships between words from massive amounts of data. Once trained, GPT can perform various language-related tasks such as writing, summarizing, answering questions and even coding all from a single model.

- **GPT**

Generative Pre-trained Transformer (GPT) is a large language model that can understand and produce human-like text. It works by learning patterns, meanings and relationships between words from massive amounts of data. Once trained, GPT can perform various language-related tasks such as writing, summarizing, answering questions and even coding all from a single model.

How GPT Works

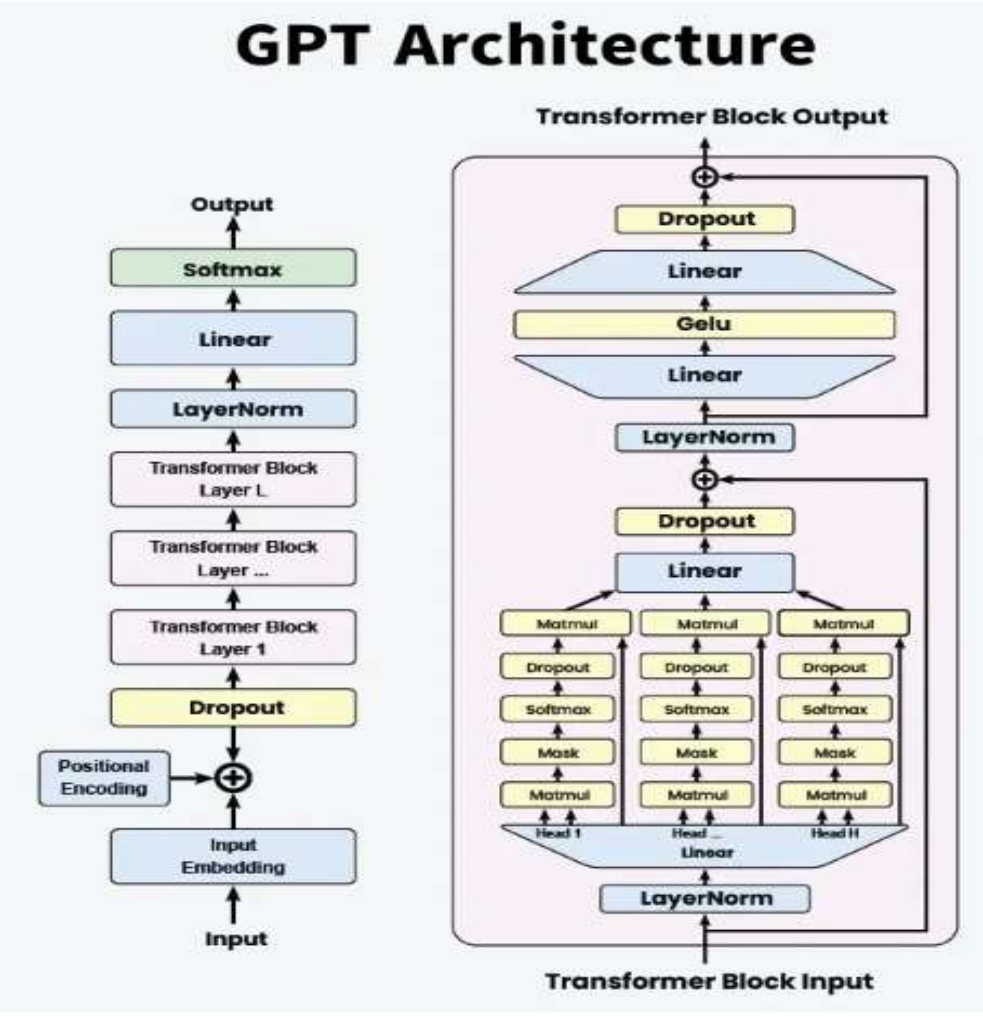
GPT models are built upon the transformer architecture, introduced in 2017, which uses self-attention mechanisms to process input data in parallel, allowing for efficient handling of long-range dependencies in text. The core process involves:

1. **Pre-training:** The model is trained on vast amounts of text data to learn language patterns, grammar, facts and some reasoning abilities.
2. **Fine-tuning:** The pre-trained model is further trained on specific datasets with human feedback to align its responses with desired outputs.

This two-step approach enables GPTs to generate coherent and contextually relevant responses across a wide array of topics and tasks.

Architecture

Let's explore the architecture:



Input Embedding

- **Input**: The raw text input is tokenized into individual tokens (words or subwords).
- **Embedding**: Each token is converted into a dense vector representation using an embedding layer.

2. Positional Encoding: Since transformers do not inherently understand the order of tokens, positional encodings are added to the input embeddings to retain the sequence information.

3. Dropout Layer: A dropout layer is applied to the embeddings to prevent overfitting during training.

4. Transformer Blocks

- **LayerNorm**: Each transformer block starts with a layer normalization.
- **Multi-Head Self-Attention**: Multi-Head Self-Attention are core component where the input passes through multiple attention heads.
- **Add & Norm**: The output of the attention mechanism is added back to the input (residual connection) and normalized again.
- **Feed-Forward Network**: A position-wise Feed-Forward Network is applied, typically consisting of two linear transformations with a GeLU activation in between.
- **Dropout**: Dropout is applied to the feed-forward network output.

5. Layer Stack: The transformer blocks are stacked to form a deeper model, allowing the network to capture more complex patterns and dependencies in the input.

6. Final Layers

- **LayerNorm**: LayerNorm is final layer normalization is applied.
- **Linear**: The output is passed through a linear layer to map it to the vocabulary size.
- **Softmax**: A Softmax layer is applied to produce the final probabilities for each token in the vocabulary.

Applications

The versatility of GPT models allows for a wide range of applications, including but not limited to:

- **Content Creation**: GPT can generate articles, stories and poetry, assisting writers with creative tasks.
- **Customer Support**: Automated chatbots and virtual assistants powered by GPT provide efficient and human-like customer service interactions.
- **Education**: GPT models can create personalized tutoring systems, generate educational content and assist with language learning.
- **Programming**: GPT's ability to generate code from natural language descriptions aids developers in software development and debugging.
- **Healthcare**: Applications include generating medical reports, assisting in research by summarizing scientific literature and providing conversational agents for patient support.

Advantages

- **Versatility**: Capable of handling diverse tasks with minimal adaptation.
- **Contextual Understanding**: Deep learning enables comprehension of complex text..
- **Scalability**: Performance improves with data size and model parameters.
- **Few-Shot Learning**: Learns new tasks from limited examples.
- **Creativity**: Generates novel and coherent content.

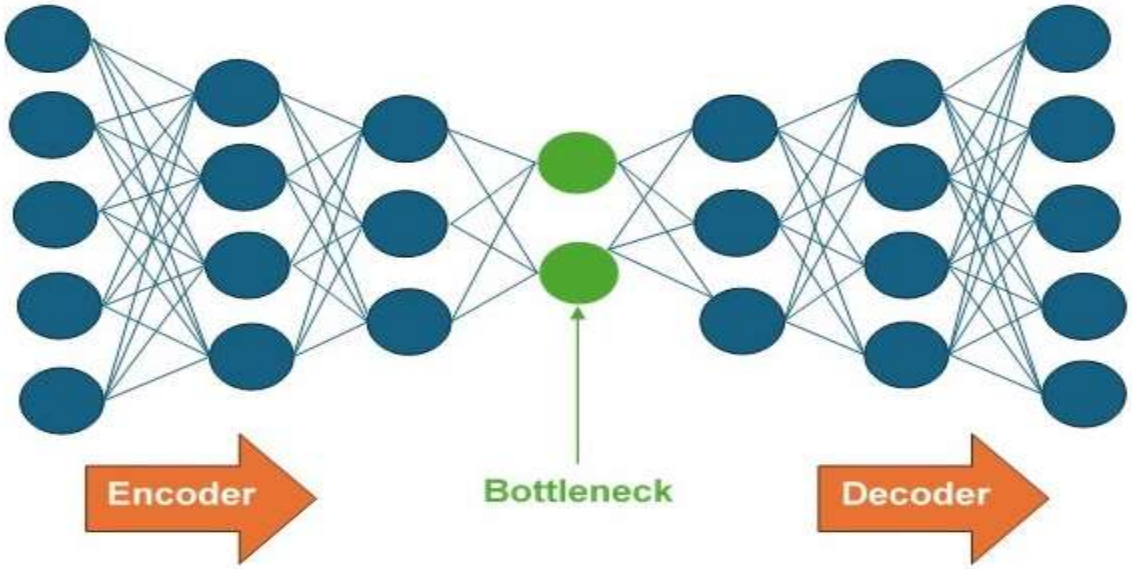
Auto Encoders

Autoencoders are an essential tool in the field of Machine Learning and Deep Learning. They are a special type of unsupervised feedforward neural network designed to learn efficient representations of the data for the purpose of dimensionality reduction, feature extraction, and generating new data.

Autoencoders consists of two components an encoder network and a decoder network. The encoder network works as a compression unit that compresses the input data into a lower-dimensional representation. On the other hand, the decoder network decompresses the compressed input data by reconstructing it. Read this chapter to understand about Autoencoders, its architecture, working, training process and hyperparameter tuning.

Autoencoders, designed for unsupervised learning, are a class of Artificial Neaural Netowrks. Like any other neural network, it consists of three different types of layers-Input, hidden and output. The number of input units in the input layer are exactly equal to the output units in the output layer. But the middle layer, i.e., the hidden layer in this network has a fewer number of units than that of input and output layers.
It first compresses the input data into a lower-dimensional representation. As the hidden layer has a lower number of units, it holds this lower-dimensional representations. Finally, at the output layer, the output is rebuilt from this reduced representation of the input.
Autoencoders are also called self-supervised ML models because they are trained as supervised ML models but while using, they work as unsupervised ML models.

Architecture of Autoencoders



Encoder − Encoder is a fully connected feed forward neural network (FFNN) thatcompresses the input data into a lower-dimensional representation.
Bottleneck layer − The bottleneck layer contains the lower-dimensionalrepresentation of the input which is to be fed into the decoder.
Decoder − Decoder is a fully connected feed forward neural network (FFNN) thatreconstruct the input back to the original dimensions.

How Do Autoencoders Work?

The principle behind the working of an autoencoders is to train the neural network toreconstruct its input data from a lower-dimensional representation. This involves twomain components: the encoder network and the decoder network

The Encoder Network

The encoder network compresses the input into a lower-dimensional representation. Thisprocess involves the following steps −

Input Layer − The input data is fed into the network through input layer.

Hidden Layes − The input data now passes through several hidden layers whereeach layer first applies a linear transformation and then a non-linear activationfunction. Each layer has fewer neurons than the previous one which graduallyreduces the dimensionality of the input data.

Bottleneck Layer (Latent Space Representation) − Bottleneck layer, the finallayer of the encoder network, stores the compressed representation of the input.This layer helps the network to learn the most essential features of the inputbecause it has a much lower dimensionality than the input data.

The Decoder Network

The decoder network reconstructs reconstruct the original input data from the lower-dimensional representation. This process is basically the reverse of the encodingprocess. It involves the following steps −

The Training Process

The training process of network to reconstruct its input data from a lower-dimensionalrepresentation involves the steps given below −

Bottleneck Layer (Latent Space Representation) − The compressed datastored by the bottleneck layer is used as the input for the decoder network.

Hidden Layes − The input data now passes through several hidden layers whereeach layer first applies a linear transformation and then a non-linear activationfunction. Each layer has more neurons than the previous one which graduallyexpanding the dimensionality of the input data back to the original input size.

Output Layer − Output layer, the final layer of the decoder network, reconstructsthe data to match the original input dimensions.

Regression Models

It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.

- Works with real-valued output variables
- Helps to identify strengths and the type of relationships
- Supports both simple and complex predictive models.
- Used for tasks like price prediction, trend forecasting and risk scoring.

Types of Regression

Regression can be classified into different types based on the number of predictor variables and the nature of the relationship between variables:

1. Simple Linear Regression

Simple Linear Regression models the relationship between one independent variable and a continuous dependent variable by fitting a straight line that minimizes the sum of squared errors. It assumes a constant rate of change, meaning the output varies proportionally with the input.

- **Application:** Estimating house price from only its size
- **Advantage:** Highly interpretable due to its simple mathematical structure
- **Disadvantage:** Cannot capture curved or complex data patterns

2. Multiple Linear Regression

Multiple Linear Regression extends simple linear regression by incorporating multiple independent variables to predict a continuous outcome. Each predictor is assigned a coefficient that reflects its individual impact while holding other variables constant.

- **Application:** Predicting house prices using multiple factors like size, location, age and number of rooms
- **Advantage:** Captures the combined influence of many factors simultaneously
- **Disadvantage:** Performance drops in the presence of multicollinearity (features highly correlated with each other)

3. Polynomial Regression

Polynomial Regression models non-linear relationships by transforming input features into higher-degree polynomial terms (e.g x^2, x^3). Although it fits non-linear curves it remains a linear model in terms of parameters.

- **Application:** Modelling curved growth trends like population increase or temperature variation
- **Advantage:** Effectively captures non-linear relationships without switching to non-linear algorithms
- **Disadvantage:** Higher-degree polynomials may lead to overfitting and unstable predictions

4. Ridge and Lasso Regression

Ridge and Lasso are regularized linear regression techniques that add penalty terms to limit large coefficients and reduce overfitting. Ridge (L2) shrinks coefficients smoothly, while Lasso (L1) can reduce some coefficients to zero, enabling feature selection.

- **Application:** Used in high-dimensional datasets like marketing attribution or gene expression data
- **Advantage:** Controls overfitting and improves generalization, especially with many predictors
- **Disadvantage:** Penalty terms make model interpretation less straightforward

5. Support Vector Regression (SVR)

Support Vector Regression applies the principles of Support Vector Machines to regression tasks. It fits a function within a defined margin (epsilon-tube) and penalizes errors only when predictions fall outside this boundary. Kernel functions allow SVR to model non-linear relationships.

- **Application:** Predicting continuous outcomes such as stock values or energy consumption
- **Advantage:** Works well with high-dimensional, complex datasets and non-linear patterns
- **Disadvantage:** Computationally intensive and requires careful tuning of kernels and parameters

6. Decision Tree Regression

Decision Tree Regression splits the data into hierarchical branches based on feature thresholds.. Each internal node represents a decision question and leaf nodes represent predicted continuous values. It learns patterns by recursively partitioning the data to minimize prediction errors.

- **Application:** Predicting customer spending behavior based on demographic and financial features
- **Advantage:** Easy to visualize and understand decision logic
- **Disadvantage:** Easily overfits, especially when the tree becomes deep and complex

7. Random Forest Regression

Random Forest Regression is an ensemble method that builds multiple decision trees using different data samples and averages their predictions. This reduces the overfitting tendency of single trees and improves accuracy through diversity (bagging). Each tree captures a slightly different aspect of the data.

- **Application:** Sales forecasting, demand planning, churn prediction
- **Advantage:** High accuracy and robust performance even on noisy datasets

- **Disadvantage:** Acts as a black-box model, making interpretation difficult due to many trees

Alright, let's unpack this without making it feel like a stats lecture.

Regression models are kind of the quiet backbone of generative AI. Even though we talk a lot about transformers and diffusion these days, regression is still everywhere under the hood.What regression means in this context

At its core, regression = predicting a continuous value.In generative AI, that usually means:

Predicting the next thing (token, pixel value, noise level, embedding, etc.)
Minimizing some notion of error between what the model predicts and what actually exists So instead of "class A or B," it's:

Given this context, what should the value be?

Where regression shows up in generative AI

1. Language models (LLMs)

Even though LLMs feel like classification ("pick the next token"), training is effectively regression.

The model outputs a vector of numbers (logits)
Those numbers are trained to match target distributions
Loss functions like cross-entropy behave like probabilistic regression

You can think of it as:
Regressing from context → probability distribution over tokens
2. Diffusion models (images, audio, video)
This is the cleanest regression example.
Diffusion models:
Add noise to data
Train a neural network to predict the noise (a continuous value)
Generation = repeatedly regressing noise away
Mathematically:
model(x_t, t) ≈ ε
That's straight-up regression in high dimensions.
3. Variational Autoencoders (VAEs)
VAEs use regression in two places:
Encoder regresses input → latent distribution parameters (mean, variance)
Decoder regresses latent → reconstructed data
The loss is usually:
econstruction loss (MSE or similar)
KL divergence regularization
4. GANs (yes, even GANs)
he generator:

Regresses from random noise → realistic samples
The discriminator:
Often trained with regression-style losses (least-squares GAN, Wasserstein GAN)
Even adversarial training doesn't escape regression math
Common regression losses in generative models
Mean Squared Error (MSE)
  Used in diffusion, VAEs, audio generation
Mean Absolute Error (MAE / L1)
  Encourages sharper outputs
Negative Log-Likelihood (NLL)
  Probabilistic regression
Cross-entropy
  Regression on probability distributions
Why regression is so important for generation
Generation is about:
Learning smooth mappings
Capturing uncertainty
Producing continuous structure
Regression excels at:
Modeling gradients
Learning fine-grained structure
Scaling to massive output spaces
That's why almost every modern generative model can be framed as:
"A very fancy regression problem with a clever objective."

Prompt Engineering

Prompt Engineering: Designing Prompts, Revising Prompts using Reinforcement Learning from Human Feedback (RLHF), Retrieval Augmented Generation, Multimodal LLM, Issues of LLM like hallucination.

Prompt Engineering

AI prompt engineering is a specific area within artificial intelligence (AI) that focuses on designing, refining, and optimizing prompts, i.e., inputs or instructions, to enable efficient and effective communication with AI models, especially large language models (LLMs) like GPT-4. The goal is that AI systems generate outputs that are accurate, relevant, and aligned with user objectives. Prompt engineering involves creating input instructions or queries to guide AI systems in producing desired outputs or responses. These prompts serve as a connection between the intentions of humans and the comprehension of machines. This allows AI models to generate outputs that are more precise and realistic. AI Prompt Engineering is essential for multiple AI applications such as natural language processing, conversational agents and content generation.

Prompt engineering is the process of creating clear and effective prompts that guide AI models to generate accurate responses. It mainly focuses on writing smart prompts for text-based AI tasks, especially NLP, to help the user and the model produce the required output.

How Prompt Engineering Works?

Imagine you're instructing a very talented but inexperienced assistant. You want them to complete a task effectively, so you need to provide clear instructions. Prompt engineering is similar - it's about crafting the right instructions, called prompts, to get the desired results from a large language model (LLM).
Working of Prompt Engineering Involves:

- **Crafting the Prompt:** You design a prompt that specifies what you want the LLM to do. This can be a question, a statement, or even an example. The wording, phrasing, and context you include all play a role in guiding the LLM's response.
- **Understanding the LLM:** Different prompts work better with different LLMs. Some techniques involve giving the LLM minimal instructions (zero-shot prompting), while others provide more context or examples (few-shot prompting).
- **Refining the Prompt:** It's often a trial-and-error process. You might need to tweak the prompt based on the LLM's output to get the kind of response you're looking for.

Applications of Prompt Engineering

Prompt engineering is used most heavily in text-based modeling, especially NLP. It adds context, meaning, and relevance to prompts, helping models generate better outputs. Some key applications include:

- **Language Translation:** It is the process of translating a piece of text from one language to another using relevant language models. Relevant prompts carefully engineering with information like the required script, dialect, and other features of source and target text can help in better response from the model.
- **Question Answering Chatbots:** A Q/A bot is one of the most popular NLP categories to work on these days. It is used by institutional websites, and shopping sites among many others. Prompts on which an AI chatbot Model is trained can largely affect the kind of response a bot generates. An example of what critical information one can add in a prompt can be adding the intent and context of the query so that the bot is not confused in generating relevant answers.
- **Text Generation**: Such a task can have a multitude of applications and hence it again becomes critical to understand the exact dimension of the user's query. The text is generated for what purpose can largely change the tone, vocabulary as well as formation of the text.

What are Prompt Engineering Techniques?

The purpose of the prompt engineering is not limited to the drafting of prompts. It is a playground that has all the tools to adjust your way of working with the big language models (LLMs) with specific purposes in mind.

Foundational Techniques

Information Retrieval: This entails the creation of prompts so that the LLM can get its knowledge base and give out what is relevant.
Context Amplification: Give supplementary context to the prompt in order to direct the understanding and attention of the LLM to its output.
Summarization: Induce the LLM to generalize or write summaries about complex themes.
Reframing: Rephrase your reminder to the LLM to consider a specific style or format for the output.
Iterative Prompting: Break down the complex tasks into smaller parts and then instruct the LLM sequentially in how to achieve the end result.

Designing Prompts in Prompt Engineering

Prompt designing is the process of creating clear, structured, and effective instructions for a generative AI model to obtain accurate, relevant, and desired outputs. A well-designed prompt acts as a bridge between human intent and machine understanding.

1. Clarity and Specificity

A prompt should be clear and unambiguous. Vague prompts often lead to generic or incorrect responses. Clearly stating what is required, such as the topic, depth, format, or constraints, helps the model generate focused results. For example, specifying "write a brief note" or "explain with an example" improves output quality.

2. Context Setting
Providing proper context helps the model understand the background of the task. Context may include the domain, audience level, or purpose of the response. For instance, mentioning "for undergraduate students" or "in simple terms" guides the model to adjust the complexity of the output.

3. Instruction-Based Prompts
Effective prompts include explicit instructions such as *explain*, *summarize*, *compare*, or *list steps*. Instruction-based prompts reduce confusion and improve task completion accuracy. Structured instructions help the model follow a logical flow.

4. Use of Examples (Few-Shot Prompting)
Including examples in the prompt helps the model learn the expected pattern or style. This technique, known as few-shot prompting, improves consistency and accuracy, especially for complex or formatted outputs.

5. Output Format Specification
Mentioning the desired output format (paragraph, points, table, code, etc.) ensures that the response matches user expectations. This is important in academic, professional, or technical use cases.

6. Constraints and Limitations
Adding constraints such as word limits, tone, or scope prevents unnecessary or irrelevant content. Constraints help in generating concise and precise responses.

7. Iterative Refinement
Prompt designing is an iterative process. Based on the generated output, the prompt can be refined or rephrased to improve accuracy and relevance. Continuous refinement leads to better results over time.

Revising Prompts using Reinforcement Learning from Human Feedback (RLHF)

Reinforcement learning from human feedback (RLHF) is a machine learning (ML) technique that uses human feedback to optimize ML models to self-learn more efficiently. Reinforcement learning (RL) techniques train software to make decisions that maximize rewards, making their outcomes more accurate. RLHF incorporates human feedback in the rewards function, so the ML model can perform tasks more aligned with human goals, wants, and needs. RLHF is used throughout generative artificial intelligence (generative AI) applications, including in large language models (LLM) The applications of artificial intelligence (AI) are broad-ranging,

from self-driving cars to natural language processing (NLP), stock market predictors, and retail personalization services. No matter the given application, the goal of AI is ultimately to mimic human responses, behaviors, and decision-making. The ML model must encode human input as training data so that the AI mimics humans more closely when completing complex tasks. RLHF is a specific technique that is used in training AI systems to appear more human, alongside other techniques such as supervised and unsupervised learning. First, the model's responses are compared to the responses of a human. Then a human assesses the quality of different responses from the machine, scoring which responses sound more human. The score can be based on innately human qualities, such as friendliness, the right degree of contextualization, and mood.

RLHF is prominent in natural language understanding, but it's also used across other generative AI applications.

Reinforcement Learning from Human Feedback (RLHF) is a technique used to improve the quality of AI model outputs by incorporating human judgments into the learning process. In prompt engineering, RLHF plays a crucial role in revising and optimizing prompts so that the model generates more accurate, relevant, and human-aligned responses.

1. Initial Prompt and Model Response
The process begins by providing an initial prompt to the AI model. The model generates a response based on its current training and understanding. This response may be partially correct, vague, or misaligned with user expectations.

2. Human Evaluation of Responses
Human evaluators review the model's responses and provide feedback. This feedback may include ranking multiple responses, marking errors, or indicating preferences based on correctness, clarity, usefulness, and safety.

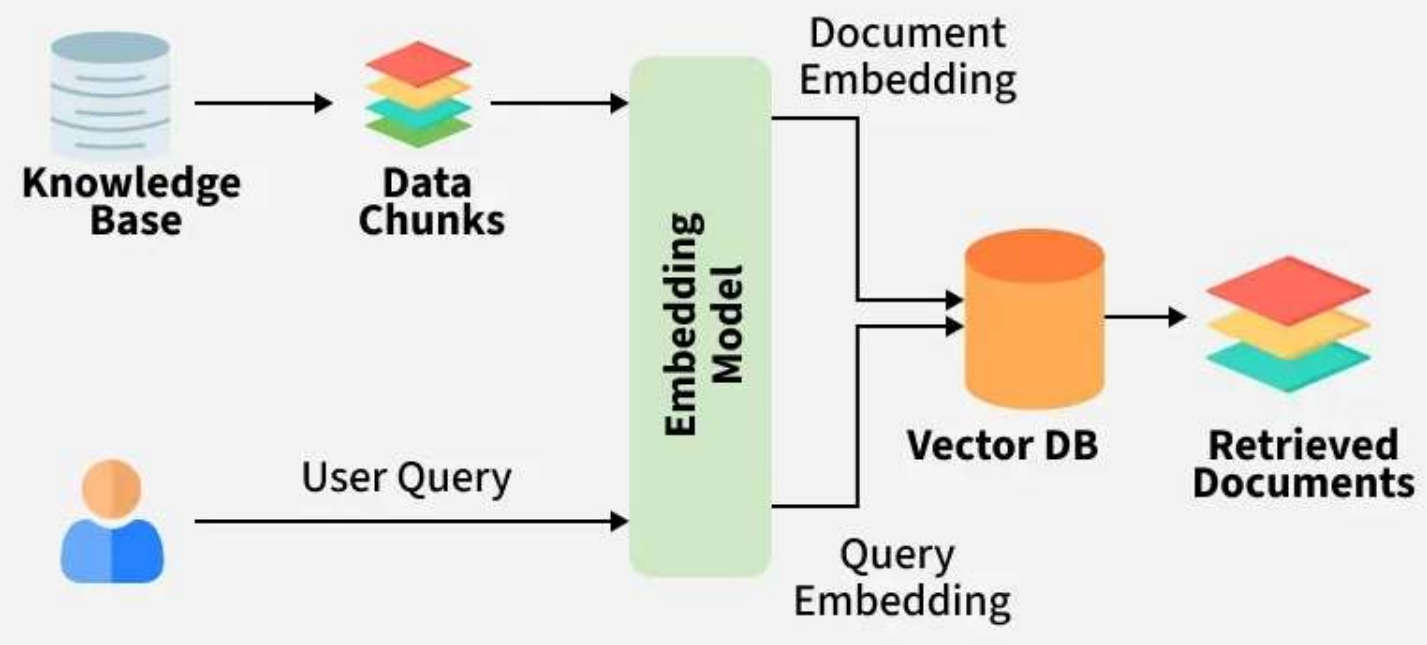
3. Reward Modeling
The human feedback is used to train a reward model. This reward model assigns higher scores to preferred responses and lower scores to poor or incorrect ones. The reward model acts as a guide for improving future outputs.

4. Reinforcement Learning Optimization
Using reinforcement learning algorithms, the base AI model is fine-tuned to maximize the reward score. During this stage, the model learns which types of responses are more acceptable and aligned with human expectations.

5. Prompt Revision and Refinement
Based on the improved model behavior, prompts are revised to be more effective. Ambiguous prompts are clarified, missing context is added, and instructions are refined to achieve consistent and high-quality outputs.

6. Iterative Feedback Loop
RLHF operates in an iterative cycle. Prompts are tested again with the updated model, human feedback is collected, and further refinements are made. Over time, this continuous loop significantly improves prompt effectiveness.

Retrieval-Augmented Generation (RAG)



Retrieval-Augmented Generation (RAG) is an advanced AI framework that combines information retrieval with text generation models like GPT to produce more accurate and up-to-date responses. Instead of relying only on pre-trained data like traditional language models, RAG fetches relevant documents from an external knowledge source before generating an answer.

1. **Access to Updated Knowledge:** LLMs are trained on fixed datasets but RAG allows them to fetch fresh and real time information from external sources.
2. **Improved Accuracy:** It reduces hallucinations in LLMs and makes answers more factually correct.
3. **Domain Specific Expertise: It l**ets us use specialized datasets like medical records and legal documents to get expert-level responses without retraining the model.
4. **Cost Efficiency:** Instead of retraining massive LLMs with new data, we simply update the external knowledge base hence saving time and resources.
5. **Personalization:** RAG can retrieve user specific information like past interactions or personal data to provide more tailored and relevant responses.

Components of RAG

The main components of RAG are:

1. **External Knowledge Source:** Stores domain specific or general information like documents, APIs or databases.
2. **Text Chunking and Preprocessing:** Breaks large text into smaller, manageable chunks and cleans it for consistency.
3. **Embedding Model:** Converts text into numerical vectors that capture semantic meaning.
4. **Vector Database:** Stores embeddings and enables similarity search for fast information retrieval.
5. **Query Encoder:** Transforms the user's query into a vector for comparison with stored embeddings.
6. **Retriever:** Finds and returns the most relevant chunks from the database based on query similarity.
7. **Prompt Augmentation Layer:** Combines retrieved chunks with the user's query to provide context to the LLM.
8. **LLM (Generator):** Generates a grounded response using both the query and retrieved knowledge.
9. **Updater (Optional):** Regularly refreshes and re-embeds data to keep the knowledge base up to date.

Working of RAG
The system first searches external sources for relevant information based on the user's query instead of relying only on existing training da

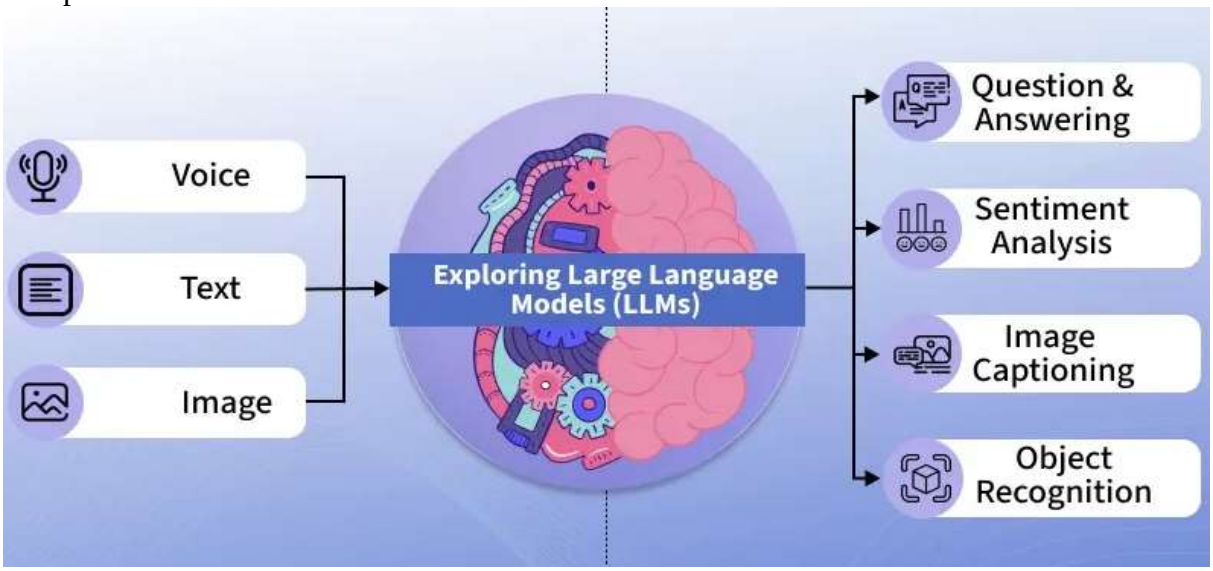
1. Creating External Data: External data from APIs, databases or documents is chunked, converted into embeddings and stored in a vector database to build a knowledge library.
2. Retrieving Relevant Information: User queries are converted into vectors and matched against stored embeddings to fetch the most relevant data ensuring accurate responses.
3. Augmenting the LLM Prompt: Retrieved content is added to the user's query giving the LLM extra context to work with.
4. Answer Generation: LLM uses both the query and retrieved data to generate a factually accurate, context aware response.
5. Keeping Data Updated: External data and embeddings are refreshed regularly in real time or scheduled so the system always retrieves latest information

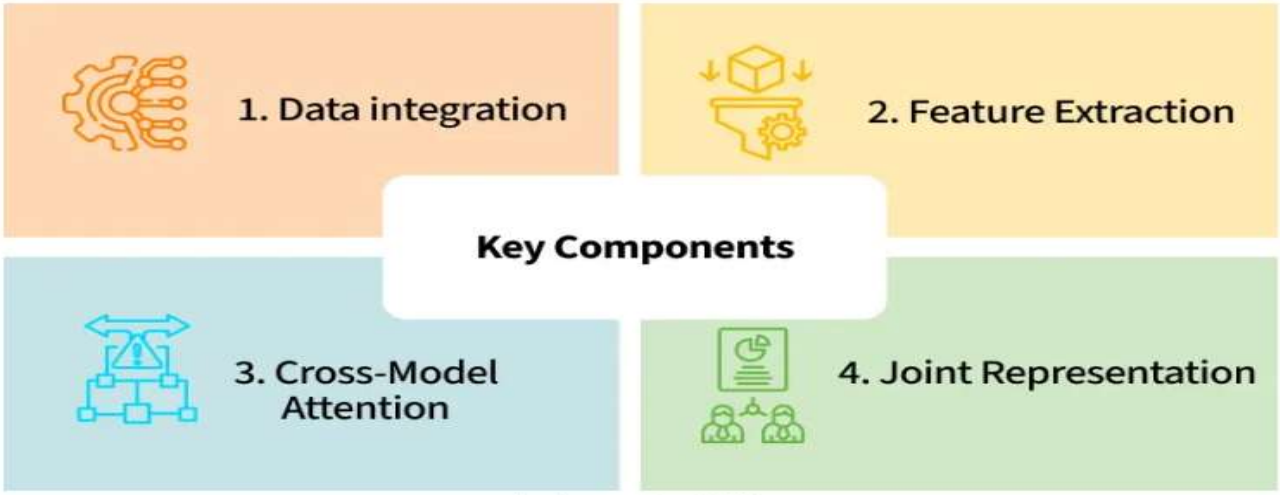
1. Creating External Data: External data from APIs, databases or documents is chunked, converted into embeddings and stored in a vector database to build a knowledge library.
2. Retrieving Relevant Information: User queries are converted into vectors and matched against stored embeddings to fetch the most relevant data ensuring accurate responses.
3. Augmenting the LLM Prompt: Retrieved content is added to the user's query giving the LLM extra context to work with.
4. Answer Generation: LLM uses both the query and retrieved data to generate a factually accurate, context aware response.
5. Keeping Data Updated: External data and embeddings are refreshed regularly in real time or scheduled so the system always retrieves latest information

---

Multimodal Large Language Models

Multimodal large language models (LLMs) integrate and process various types of data such as text, images, audio and video to enhance understanding and generate responses. For example, an MLLM can interpret a text description, analyze a corresponding image and generate a response that encompasses both forms of input. This capability allows them to perform tasks that require understanding of various types of data, making them more versatile and powerful.

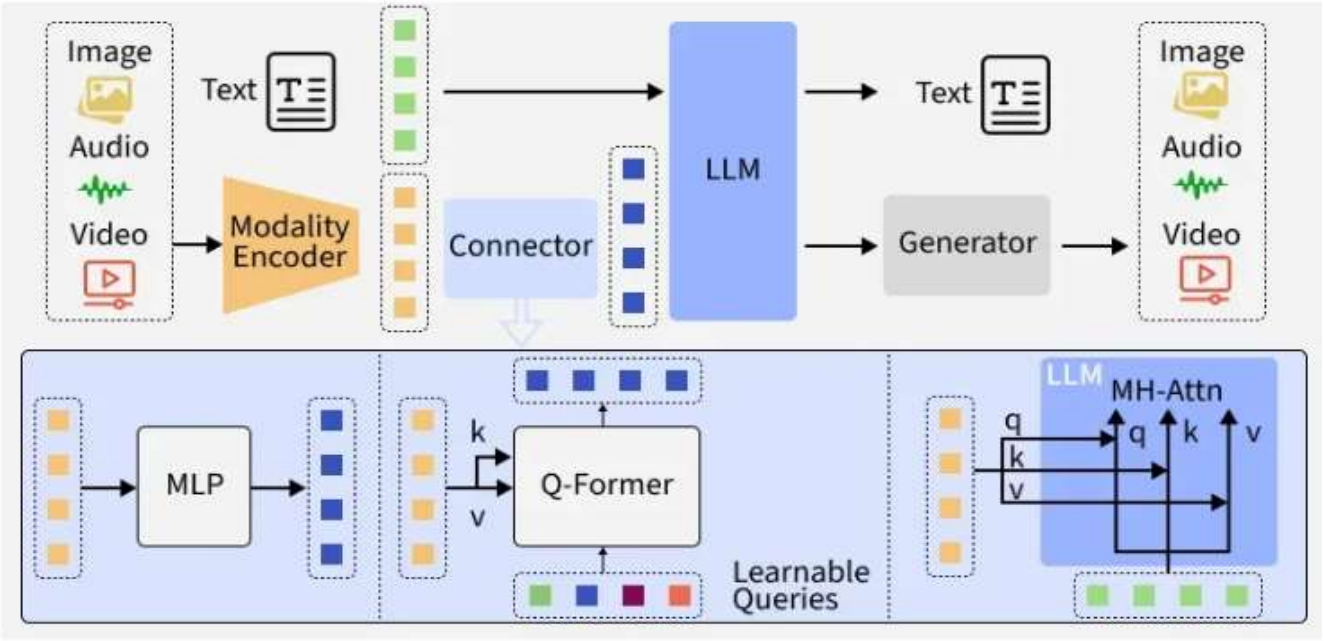




Key Components of MLLMs

- **Data Integration:** MLLMs use algorithms to combine data from multiple sources, ensuring that the information from each modality is accurately represented and integrated.
- **Feature Extraction:** The model extracts relevant features from each type of input. For example, it might identify objects and their relationships in an image while understanding the context and meaning of accompanying text.
- **Joint Representation:** By creating a joint representation of the multimodal data, the model can make inferences and generate outputs that consider all available information.
- **Cross-Modal Attention:** Techniques like cross-modal attention help the model focus on relevant parts of the data from different modalities, improving its ability to generate coherent and contextually appropriate responses.

Architecture of Multimodal Large Language Models
Multimodal models handles different types of data. Let's see the various components



Architecture of MLLM

- Modality Encoder: **Specialized neural networks (like CNNs or Vision Transformers for images, audio models for sound and LLMs for text) process each input type, image, audio, video or text and convert them into high-dimensional feature embeddings.**

- Connector (Aligner/Projector): **This module transforms and synchronizes the varied modality embeddings, adapting them so they can be effectively interpreted and used by the central LLM. The connector may use techniques like MLPs, Q-Formers or learnable queries to project modality features into a compatible space.**
- Fusion Mechanism: **Information from all input types is integrated either at the feature level (early fusion), after preliminary processing (late fusion) or at multiple layers (hybrid fusion), enabling rich, context-aware cross-modal understanding.**
- LLM Backbone: **The large language model (such as GPT-4o or Gemini) serves as the reasoning core. It attends to and uses the fused multi-modal information to generate holistic, context-driven text outputs or to assist in further generative tasks like creating images, audio or video.**
- **Popular Multimodal Large Language Models**
- **The multimodal large language models have broad applications in field such as computer vision, natural language processing and multimedia content generation. Some of the popular MLLMs are:**
- GPT-4o: **OpenAI's GPT-4o is designed to help us interact with text, images, audio and video all at once, making conversations and creative work feel smooth and natural.**
- Uses: **chatbots that see and talk, making images and stories, helping accessibility, creative content creation**
- Advantages: **real-time responses, natural conversations with emotion, supports many languages**
- Limitations: **needs powerful hardware, some features require paid access, video support still improving**
- Gemini 2.5 Pro: **Google's Gemini 2.5 Pro can handle tons of text, pictures, audio and video, letting us tackle big projects and complex conversations without missing a beat.**
- Uses: **long conversations, coding help, analyzing business data, summarizing documents**
- Advantages: **remembers a lot at once, great for logic-heavy tasks, quick performance**
- Limitations: **best features for business users, high hardware requirements, frequent updates may impact stability**
- Qwen 2.5 VL / Qwen 3: **Alibaba has built Qwen VL / Qwen 3 to easily mix language and images, so we can chat, create or get help in over 100 languages effortlessly.**
- Uses: **customer service bots, multilingual chat, creating guides, educational support**
- Advantages: **easy switching between reasoning and chat modes, strong language support, good for business and education**
- Limitations: **top features mostly inside Alibaba products, less popular outside Asia, best performance needs customization**
- Llama 4: **Meta's Llama 4 is an open-source model that lets us work with words, images and videos together, offering flexibility for a variety of needs worldwide.**
- Uses: **answering questions about pictures or documents, searching and organizing info, multilingual support**
- Advantages: **open-source for research, handles large files, highly customizable**
- Limitations: **needs strong hardware for big jobs, setup impacts quality, may require extra fine-tuning**
- Claude 3.5 Sonnet: **Created by Anthropic, this model focuses on safe reasoning with text and images.**
- Uses: **data review, making reports, understanding charts and documents, research tasks**
- Advantages: **robust safety checks, good at complex math and deep thinking, trusted by businesses**
- Limitations: **limited video support, some features paid-only, less visually creative than others**
- **Applications of Multimodal Large Language Models**
- **Let's see some applications of MLLMs,**
- Healthcare: **Analyzing scans along with patient text data for diagnostic support.**
- Education: **Interactive tutoring, explaining diagrams and aiding in language learning.**
- Creative Content: **Generating images from text prompts, captioning videos and storytelling.**
- Customer Service: **Interpreting screenshots, documents or voice queries in support workflows.**
- Accessibility: **Making digital content usable for people with various disabilities**

Issues of LLM like hallucination.

Large Language Models (LLMs) are powerful, but they come with several important limitations. **Hallucination** is one of the most discussed issues, along with a few closely related challenges. Here's a clear, exam-ready explanation.

Hallucination in LLMs

Hallucination refers to the phenomenon where an LLM generates **factually incorrect, misleading, or entirely fabricated information**, even though the response appears fluent and confident.

LLMs predict the next word based on patterns in training data; they **do not truly understand facts** or verify truth. As a result, when the model lacks sufficient knowledge or context, it may "fill the gaps" with plausible-sounding but false content.

Examples:

- Generating non-existent research papers or fake citations
- Providing incorrect historical facts
- Giving wrong technical explanations with high confidence

Reasons for hallucination:

- Training on noisy, incomplete, or outdated data
- Lack of real-time fact verification
- Over-generalization from learned patterns
- Ambiguous or poorly framed user prompts

Impact:

- Reduces trustworthiness of LLM outputs
- Dangerous in high-risk domains like healthcare, law, and finance
- Can mislead students, researchers, and decision-makers

Other Major Issues of LLMs

1. **Bias and Fairness Issues**
   LLMs may reflect social, cultural, or gender biases present in their training data, leading to unfair or discriminatory outputs.
2. **Lack of Explainability**
   LLMs operate as black-box models. They provide answers but cannot clearly explain *why* a particular answer was generated.
3. **Outdated Knowledge**
   Once trained, an LLM's knowledge is static and may not include recent events, new research, or updated policies.
4. **Overconfidence in Wrong Answers**
   Even when incorrect, LLMs often respond confidently, making it difficult for users to detect errors.
5. **Sensitivity to Prompting**
   Small changes in input phrasing can lead to very different outputs, affecting consistency and reliability.

6. **High Computational and Energy Cost**
   Training and deploying LLMs require significant computational resources, leading to high cost and environmental concerns.
7. **Security and Privacy Risks**
   LLMs may unintentionally reveal sensitive information or be exploited for generating malicious content.

- High Costs: Training requires millions of dollars in compute resources.
- Time-Intensive: Training large models can take weeks or months.
- Data Challenges: Limited availability of high-quality, legal and unbiased text data.
- Environmental Impact: High energy consumption leading to significant carbon footprint.
- Ethical Concerns: Bias, misinformation risks and responsible deployment remains a major issue.

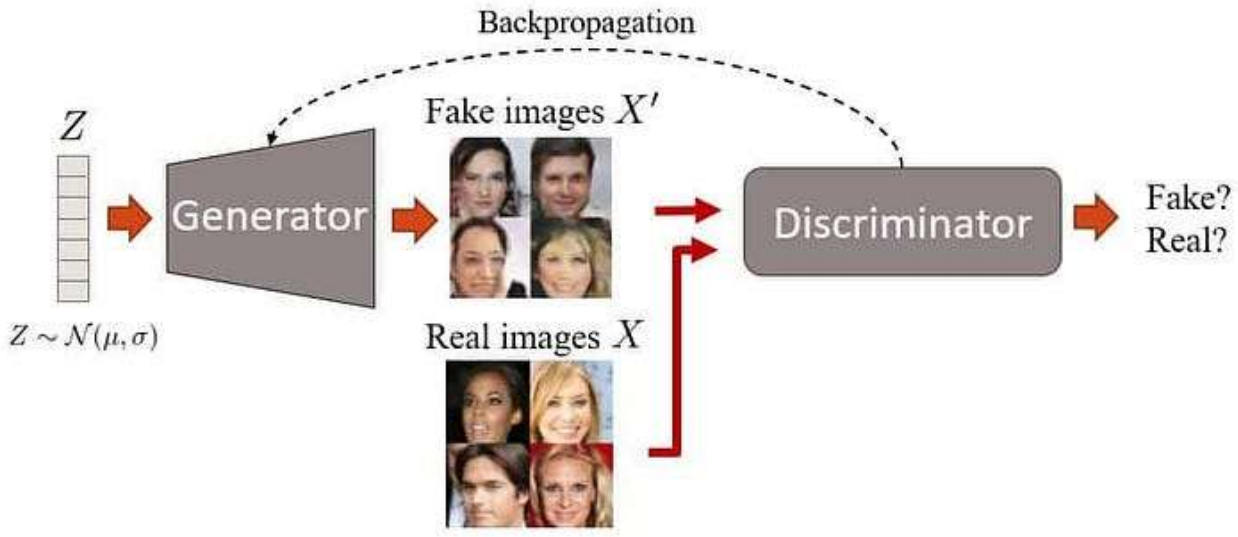
UNIT-III

Generation of Images: Introduction to Generative Adversarial Networks, Adversarial Training, Process, Nash Equilibrium, Vibrational Auto encoders, Encoder-Decoder Architectures, Stable Diffusion Models, Introduction to Transformer-based Image Generation, CLIP, Visual Transformers ViT- Dall-E2 and Dall-E3, GPT-4V, Issues of Image Generation models like Mode Collapse and Stability

GENERATION OF IMAGES

INTRODUCTION TO GENERATIVE ADVERSARIAL NETWORKS

"GAN stands for Generative Adversarial Network, and it is a class of artificial intelligence algorithms used in machine learning and deep learning for generating data. GANs were introduced by Ian Goodfellow and his colleagues in 2014 and have since become a popular and powerful tool in various applications, including image generation, text generation, and more."



Unlike traditional models that only recognize or classify data, they take a creative way by generating entirely new content that closely resembles real-world data. This ability helped various fields such as art, gaming, healthcare and data science. In this article, we will see more about GANs and its core concepts.

Architecture of GAN

GAN consist of two main models that work together to create realistic synthetic data which are as follows:

1. Generator Model

The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through backpropagation. Its objective is to produce samples that the discriminator classifies as real.

Generator Loss Function: The generator tries to minimize this loss:

$$J_G = -\frac{1}{m}\sum_{i=1}^{m} \log D(G(z_i))$$

where

JG measure how well the generator is fooling the discriminator.
G(zi) is the generated sample from random noise zi
D(G(zi)) is the discriminator's estimated probability that the generated sample is real.

2. Discriminator Model

The discriminator acts as a binary classifier helps in distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately. When dealing with image data, the discriminator uses convolutional layers or other relevant architectures which help to extract features and enhance the model's ability.

Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m}\sum_{i=1}^{m} \log D(x_i) - \frac{1}{m}\sum_{i=1}^{m} \log(1 - D(G(z_i)))$$

- JD measures how well the discriminator classifies real and fake samples.
- xi is a real data sample.
- G(zi) is a fake sample from the generator.
- D(xi) is the discriminator's probability that xi is real.
- D(G(zi)) is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize logD(xi) and fake data as fake (maximize log(1−D(G(zi)))

MinMax Loss

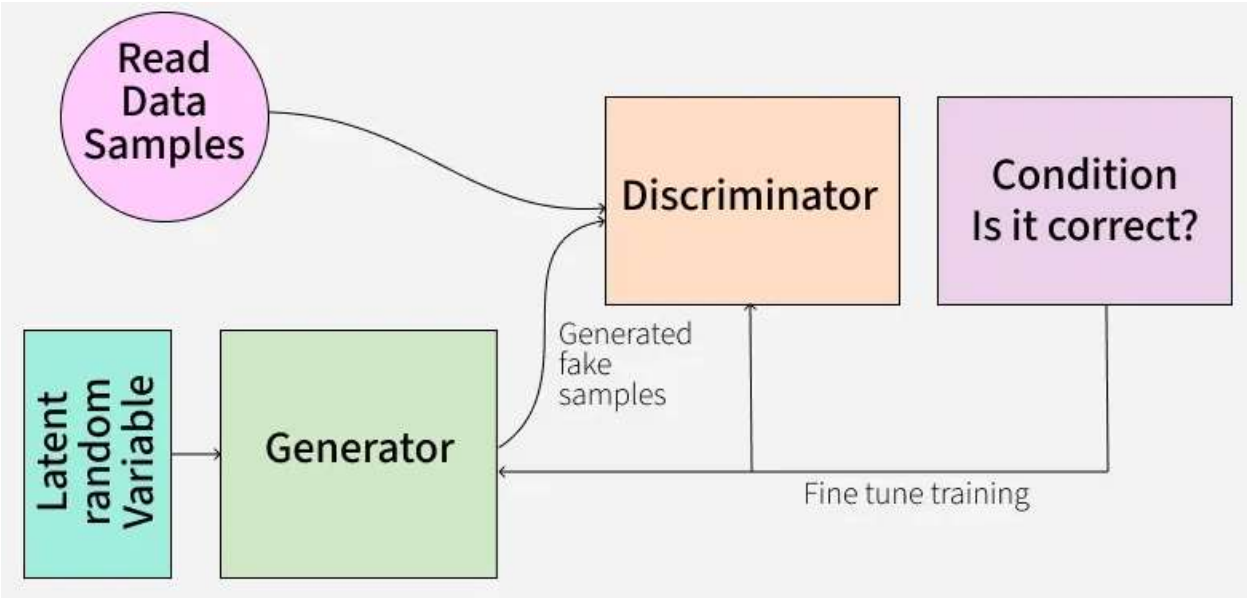
GANs are trained using a MinMax Loss between the generator and discriminator:

$$min_G\ max_D(G, D) = [\mathbb{E}_{x \sim p_{data}}[log\ D(x)] + \mathbb{E}_{z \sim p_z(z)}[log(1 - D(g(z)))]$$

where,

- G is generator network and is D is the discriminator network
- $p_{data}(x)$ = true data distribution
- $p_z(z)$= distribution of random noise (usually normal or uniform)
- $D(x)$ = discriminator's estimate of real data
- $D(G(z))$= discriminator's estimate of generated data

The generator tries to minimize this loss (to fool the discriminator) and the discriminator tries to maximize it (to detect fakes accurately).



How does a GAN work?

GAN train by having two networks the Generator (G) and the Discriminator (D) compete and improve together. Here's the step-by-step process

1. Generator's First Move

The generator starts with a random noise vector like random numbers. It uses this noise as a starting point to create a fake data sample such as a generated image. The generator’s internal layers transform this noise into something that looks like real data.

2. Discriminator's Turn

The discriminator receives two types of data:

Real samples from the actual training dataset.

Fake samples created by the generator.

D's job is to analyze each input and find whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 shows the data is likely real and 0 suggests it's fake.

3. Adversarial Learning

- If the discriminator correctly classifies real and fake data it gets better at its job.
- If the generator fools the discriminator by creating realistic fake data, it receives a positive update and the discriminator is penalized for making a wrong decision.

4. Generator's Improvement

- Each time the discriminator mistakes fake data for real, the generator learns from this success.
- Through many iterations, the generator improves and creates more convincing fake samples.

5. Discriminator's Adaptation

- The discriminator also learns continuously by updating itself to better spot fake data.
- This constant back-and-forth makes both networks stronger over time.

6. Training Progression

- As training continues, the generator becomes highly proficient at producing realistic data.

- Eventually the discriminator struggles to distinguish real from fake shows that the GAN has reached a well-trained state.
- At this point, the generator can produce high-quality synthetic data that can be used for different applications.

Types of GAN

There are several types of GANs each designed for different purposes. Here are some important types:

1. Vanilla GAN

Vanilla GAN is the simplest type of GAN. It consists of:

- A generator and a discriminator both are built using multi-layer perceptrons (MLPs).
- The model optimizes its mathematical formulation using stochastic gradient descent (SGD).

While foundational, Vanilla GAN can face problems like:

- **Mode collapse**: The generator produces limited types of outputs repeatedly.
- **Unstable training**: The generator and discriminator may not improve smoothly.

2. Conditional GAN (CGAN)

Conditional GAN (CGAN) adds an additional conditional parameter to guide the generation process. Instead of generating data randomly they allow the model to produce specific types of outputs.

Working of CGANs:

- A conditional variable (y) is fed into both the generator and the discriminator.
- This ensures that the generator creates data corresponding to the given condition (e.g generating images of specific objects).
- The discriminator also receives the labels to help distinguish between real and fake data.

Example: Instead of generating any random image, CGAN can generate a specific object like a dog or a cat based on the label.

3. Deep Convolutional GAN (DCGAN)

Deep Convolutional GAN (DCGAN) are among the most popular types of GANs used for image generation.

They are important because they:

- Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).
- Max pooling layers are replaced with convolutional stride helps in making the model more efficient.
- Fully connected layers are removed, which allows for better spatial understanding of images.

DCGANs are successful because they generate high-quality, realistic images.

4. Laplacian Pyramid GAN (LAPGAN)

Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by using a multi-resolution approach.

Working of LAPGAN:

Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid. Images are first down sampled at each layer of the pyramid and upscaled again using Conditional GAN (CGAN).

This process allows the image to gradually refine details and helps in reducing noise and improving clarity. Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for photorealistic image generation.

5. Super Resolution GAN (SRGAN)

Super-Resolution GAN (SRGAN) is designed to increase the resolution of low-quality images while preserving details.

Working of SRGAN:

Uses a deep neural network combined with an adversarial loss function. Enhances low-resolution images by adding finer details helps in making them appear sharper and more realistic. Helps to reduce common image upscaling errors such as blurriness and pixelation.

ADVERSARIAL TRAINING PROCESS:-

Adversarial Training is the core learning mechanism used in image generation models, especially in Generative Adversarial Networks. It is a competitive learning process between two neural networks that improves image quality through continuous feedback.

1. Introduction

Adversarial training is a learning strategy where two models compete with each other:

1. Generator (G) – Generates fake images.
2. Discriminator (D) – Distinguishes between real and fake images.

The goal of adversarial training is to make the generated images so realistic that the discriminator cannot differentiate them from real images.

2. Basic Architecture

The adversarial training process consists of two deep neural networks:
(i) Generator Network (G)
Takes random noise vector (z) as input.
Produces synthetic image as output.
Learns to mimic the real image distribution.
(ii) Discriminator Network (D)
Takes an image as input.
Outputs probability whether the image is real (1) or fake (0).
Acts as a binary classifier.
Both networks are trained simultaneously in a competitive manner.

3. Step-by-Step Adversarial Training Process

The adversarial training process in image generation follows these steps:

Step 1: Initialize Networks

Randomly initialize Generator and Discriminator parameters.

Select loss function (Binary Cross Entropy commonly used).

Step 2: Train Discriminator

1. Take a batch of real images from dataset.
2. Label them as Real (1).
3. Generate fake images using Generator from random noise.
4. Label them as Fake (0).
5. Compute Discriminator loss:

Loss on real images.

Loss on fake images.
6. Update Discriminator weights using backpropagation.

Objective: Maximize ability to distinguish real vs fake.
Step 3: Train Generator

1. Generate fake images using random noise.
2. Pass fake images to Discriminator.
3. Compute Generator loss based on how well it fools the Discriminator.
4. Update Generator weights.

Objective: Minimize Discriminator's ability to detect fake images.
Step 4: Iterative Optimization
Repeat Step 2 and Step 3 alternately.
Continue until equilibrium is reached.
At equilibrium, generated images look highly realistic.
4. Mathematical Objective Function

The adversarial loss is defined as:

$$\min_G \max_D V(D,G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log (1 - D(G(z)))]$$

Process of Generative Adversarial Networks (GANs) for Image Generation:

STEP 1: IMPORT NECESSARY LIBRARIES AND LOAD DATASET

We will be using TensorFlow, Keras, NumPy and Matplotlib.

```
import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models, optimizers

import matplotlib.pyplot as plt
```

STEP 2: DATASET PREPARATION

Prepare the MNIST data by reshaping images to the required format and normalizing pixel values to the range [0,1]. Normalization helps stabilize training by keeping input values within a small range.

```
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

x_train = x_train.reshape((-1, 28, 28, 1)).astype('float32') / 255.0
```

STEP 3: BUILDING THE MODELS

Define the architectures of the generator and discriminator using convolutional neural networks (CNNs) designed to efficiently process and generate images.
Generator Model with CNN Layers:

- **Dense Layer:** Converts the 100-dimensional noise vector into a high-dimensional feature map.
- **Reshape**: Transforms the feature map into a 3D tensor suitable for convolutional processing.
- **Conv2DTranspose Layers**: Perform upsampling and convolution simultaneously helps in gradually increasing image resolution.
- **BatchNormalization**: Stabilizes training and speeds convergence.
- **Activation Functions**: ReLU for hidden layers and sigmoid in the output layer to constrain pixel values between 0 and 1.

Discriminator Model with CNN Layers:

- **Conv2D Layers:** Apply stride-2 convolutions to downsample images helps in reducing dimensionality and increasing receptive fields.
- **BatchNormalization**: Helps to maintain stable training.
- **Flatten**: Converts feature maps into 1D vectors for classification.
- **Dense Output Layer**: Outputs a single probability indicating whether an image is real or fake.

```
def build_discriminator_cnn():
    model = models.Sequential([

        layers.Conv2D(64, kernel_size=3, strides=2, input_shape=(28, 28, 1),
padding='same', activation='relu'),

        layers.Conv2D(128, kernel_size=3, strides=2, padding='same', activation='relu'),
        layers.BatchNormalization(),

        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ])
    return model
```

STEP 4: COMPILING THE MODELS

Compile the GAN by connecting the generator and discriminator. During generator training, the discriminator's weights are frozen to prevent updates.

- **discriminator_cnn.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01), loss='binary_crossentropy', metrics=['accuracy']):** Compiles the discriminator with Adam optimizer, binary cross-entropy loss, and accuracy metric.
- **gan_input = layers.Input(shape=(100,))**: Defines the input layer for the GAN, expecting a 100-dimensional noise vector.

```
generator_cnn = build_generator_cnn()
discriminator_cnn = build_discriminator_cnn()
```

```
discriminator_cnn.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
                          loss='binary_crossentropy',
                          metrics=['accuracy'])

discriminator_cnn.trainable = False

gan_input = layers.Input(shape=(100,))
gan_output = discriminator_cnn(generator_cnn(gan_input))
gan_cnn = models.Model(gan_input, gan_output)
gan_cnn.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
                loss='binary_crossentropy')
```

STEP 5: MODEL TRAINING AND VISUALIZING

Train the GAN by alternating between:

- Training the discriminator to correctly classify real and fake images.
- Training the generator to fool the discriminator by producing more realistic images.

Visualize generated images periodically to monitor training progress.

- **idx = np.random.randint(0, x_train.shape[0], batch_size)**: Randomly selects indices to sample a batch of real images from the training data.
- **g_loss = gan_cnn.train_on_batch(noise, valid_labels)**: Trains the GAN model (generator + frozen discriminator) to improve generator performance.

```
epochs = 100000
batch_size = 64

for epoch in range(epochs+1):

    noise = np.random.normal(0, 1, (batch_size, 100))
    generated_images = generator_cnn.predict(noise)

    idx = np.random.randint(0, x_train.shape[0], batch_size)
    real_images = x_train[idx]

    real_labels = np.ones((batch_size, 1))
    fake_labels = np.zeros((batch_size, 1))

    d_loss_real = discriminator_cnn.train_on_batch(real_images, real_labels)
    d_loss_fake = discriminator_cnn.train_on_batch(generated_images, fake_labels)
    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
```

```
    noise = np.random.normal(0, 1, (batch_size, 100))
    valid_labels = np.ones((batch_size, 1))
    g_loss = gan_cnn.train_on_batch(noise, valid_labels)

    if epoch % 100 == 0:
        print(f"Epoch {epoch}: D Loss: {d_loss[0]}, G Loss: {g_loss}")

    if epoch % 1000 == 0:
        test_noise = np.random.normal(0, 1, (1, 100))
        test_img = generator_cnn.predict(test_noise)[0].reshape(28, 28)
        plt.imshow(test_img, cmap='gray')
        plt.axis('off')
        plt.show()
```

NASH EQUILIBRIUM

A Nash Equilibrium occurs when:

Each player's strategy is the best response to the strategies chosen by other players.

In simple words:

- No player has an incentive to deviate.
- All players are optimizing given others' decisions.
- The system becomes stable.

Nash Equilibrium is a fundamental concept in Game Theory proposed by **John Nash**. It describes a stable state in a strategic game where no player can gain advantage by unilaterally changing their strategy while other players keep their strategies unchanged.

It is widely used in economics, political science, artificial intelligence, and adversarial training models like **Generative Adversarial Networks**.

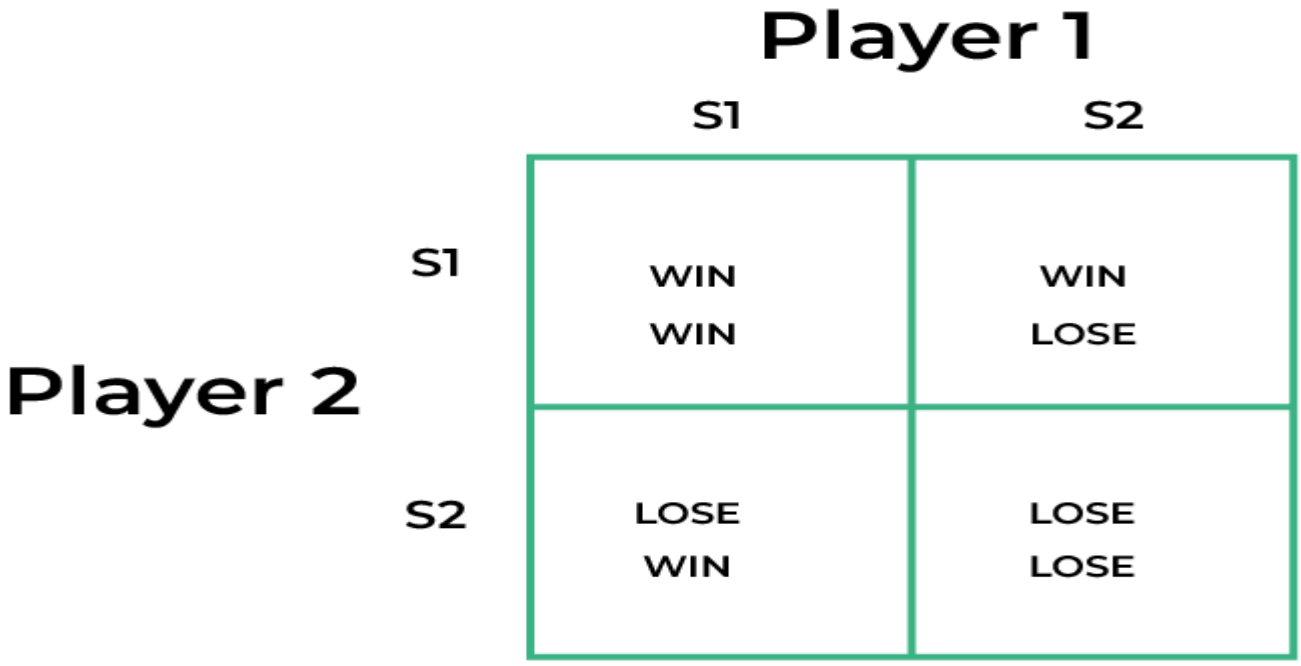
Nash equilibrium is an important concept in **game theory that provides the optimal outcome** in case the player **doesn't deviate** from their initial strategy. This is done in response to no incentive provided to the players for such deviation. This was named after the Mathematician, John Nash, defining the solution of a non-cooperative game involving two or more players. Because the strategy remains optimal and the user doesn't receive any incentive also denotes that the players are aware of each other's strategy and so will not deviate at all. Considering the other player remains constant in their strategy, an individual can receive no incremental benefit from such a deviation. **A game can have multiple or none Nash equilibrium.**

Key Points

1. Nash equilibrium provides an **optimal solution** to encounter the required outcome by not deviating from their initial strategy.
2. Since individuals are already aware of each other's strategy, **both the players win** as everyone gets the outcome that they thought for.
3. One such example is the **Prisoner's dilemma**.
4. Since the players quest to win, they would be performing that strategy that would lead them to such a state. Thus the chosen strategy is the best and the optimal solution that they can use. This is also in **conjunction with the dominant strategy**.
5. Further, as stated there cannot be any Nash equilibrium in a game. So, **it is not always true that the strategy chosen is the optimal one**

EXAMPLE OF NASH EQUILIBRIUM

In the given game, we have two players with strategies S1 and S2.



S1 being the optimal strategy considering both the players know each other's approach or they are made aware of it and both will be moving their initial strategy. They will opt for it because they know deviating from it will not be in the interest of the match.

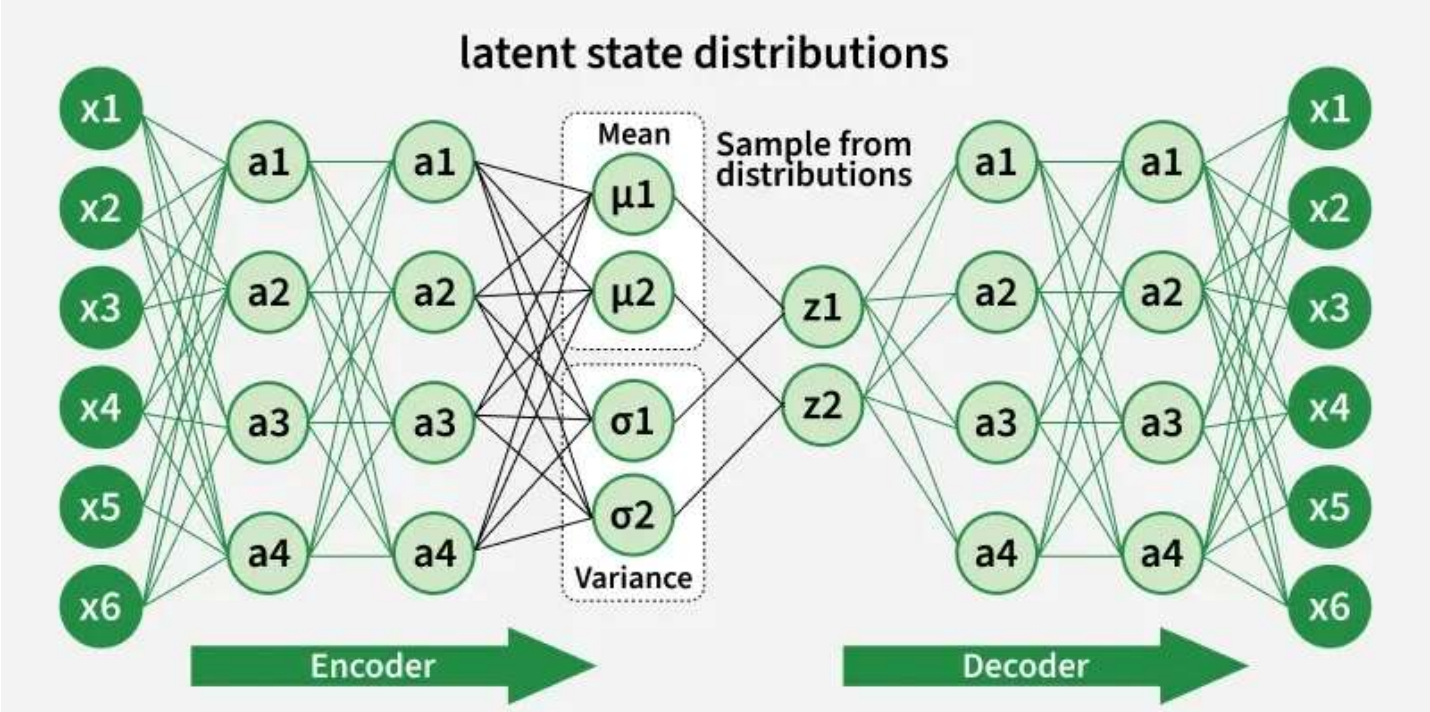
Hence in case they opt for S1, both will win. In other cases, deviating to S2 will result in defeat of one of them.

Variational AutoEncoders

Variational Autoencoders (VAEs) are generative models that learn a smooth, probabilistic latent space, allowing them not only to compress and reconstruct data but also to generate entirely new, realistic samples. VAEs capture the underlying structure of a dataset and produce outputs that closely resemble the original data.

- Learns a continuous latent representation
- Enables controlled and meaningful data generation
- Widely used in image synthesis, anomaly detection, and representation learning

Architecture of Variational Autoencoder



VAE is a special kind of autoencoder that can generate new data instead of just compressing and reconstructing it. It has three main parts:

1. Encoder (Understanding the Input)

The encoder takes input data like images or text and learns its key features. Instead of outputting one fixed value, it produces two vectors for each feature:

- **Mean (μ):** A central value representing the data.
- **Standard Deviation (σ):** It is a measure of how much the values can vary.

These two values define a range of possibilities instead of a single number.

2. Latent Space (Adding Some Randomness)

Instead of encoding the input as one fixed point it pick a random point within the range given by the mean and standard deviation. This randomness lets the model create slightly different versions of data which is useful for generating new, realistic samples.

3. Decoder (Reconstructing or Creating New Data)

The decoder takes the random sample from the latent space and tries to reconstruct the original input. Since the encoder gives a range, the decoder can produce new data that is similar but not identical to what it has seen.

Mathematics behind Variational Autoencoder

Variational autoencoder uses KL-divergence as its loss function the goal of this is to minimize the difference between a supposed distribution and original distribution of dataset.

Suppose we have a distribution z and we want to generate the observation x from it. In other words we want to calculate $p(z|x)$ We can do it by following way:

$$p\left(z|x\right) = \frac{p(x|z)p(z)}{p(x)}$$

But, the calculation of $p(x)$ can be difficult:

$$p\left(x\right) = \int p\left(x|z\right) p\left(z\right) dz$$

This usually makes it an intractable distribution. Hence we need to approximate $p(z|x)$ to $q(z|x)$ to make it a tractable distribution. To better approximate $p(z|x)$ to $q(z|x)$ we will minimize the KL-divergence loss which calculates how similar two distributions are:

$$\min KL\left(q\left(z|x\right)||p\left(z|x\right)\right)$$

By simplifying the above minimization problem is equivalent to the following maximization problem :

$$E_{q(z|x)} \log p\left(x|z\right) - KL\left(q\left(z|x\right)||p\left(z\right)\right)$$

The first term represents the reconstruction likelihood and the other term ensures that our learned distribution q is similar to the true prior distribution p. Thus our total loss consists of two terms one is reconstruction error and other is KL divergence loss:

$$Loss = L\left(x, \hat{x}\right) + \sum_{j} KL\left(q_j\left(z|x\right)||p\left(z\right)\right)$$

Implementing Variational Autoencoder

We will build a Variational Autoencoder using TensorFlow and Keras. The model will be trained on the Fashion-MNIST dataset which contains 28×28 grayscale images of clothing items. This dataset is available directly through Keras.

Step 1: Importing Libraries

First we will be importing Numpy, TensorFlow, Keras layers and Matplotlib for this implementation.

```
import numpy as np

import tensorflow as tf

import keras

from keras import layers
```

Step 2: Creating a Sampling Layer

The sampling layer acts as the bottleneck, taking the mean and standard deviation from the encoder and sampling latent vectors by adding randomness. This allows the VAE to generate varied outputs.

- **epsilon = tf.random.normal(shape=tf.shape(mean))**: Generate random noise from normal distribution.
- **return mean + tf.exp(0.5 * log_var) * epsilon**: Apply reparameterization trick to sample latent vector.

```
class Sampling(layers.Layer):

    """Uses (mean, log_var) to sample z, the vector encoding a digit."""



    def call(self, inputs):

        mean, log_var = inputs

        batch = tf.shape(mean)[0]
```

```
        dim = tf.shape(mean)[1]

        epsilon = tf.random.normal(shape=(batch, dim))

        return mean + tf.exp(0.5 * log_var) * epsilon
```

Step 3: Defining Encoder Block

The encoder takes input images and outputs two vectors: mean and log variance. These describe the distribution from which latent vectors are sampled.

- **x = layers.Dense(128, activation="relu")(x)**: Fully connected layer with 128 units and ReLU activation.
- **encoder = keras.Model(encoder_inputs, [mean, log_var, z], name="encoder")**: Define encoder model from input to outputs.

```
latent_dim = 2


encoder_inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(64, 3, activation="relu", strides=2,
                  padding="same")(encoder_inputs)
x = layers.Conv2D(128, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Flatten()(x)
x = layers.Dense(16, activation="relu")(x)
mean = layers.Dense(latent_dim, name="mean")(x)
log_var = layers.Dense(latent_dim, name="log_var")(x)
z = Sampling()([mean, log_var])
encoder = keras.Model(encoder_inputs, [mean, log_var, z], name="encoder")
encoder.summary()
```

Output:

Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0	-
conv2d_2 (Conv2D)	(None, 14, 14, 64)	640	input_layer_1[0][0]
conv2d_3 (Conv2D)	(None, 7, 7, 128)	73,856	conv2d_2[0][0]
flatten_1 (Flatten)	(None, 6272)	0	conv2d_3[0][0]
dense_1 (Dense)	(None, 16)	100,368	flatten_1[0][0]
mean (Dense)	(None, 2)	34	dense_1[0][0]
log_var (Dense)	(None, 2)	34	dense_1[0][0]
sampling_1 (Sampling)	(None, 2)	0	mean[0][0], log_var[0][0]

Total params: 174,932 (683.33 KB)
Trainable params: 174,932 (683.33 KB)
Non-trainable params: 0 (0.00 B)

Step 4: Defining Decoder Block

Now we will define the architecture of decoder part of our autoencoder which takes sampled latent vectors and reconstructs the image.

- **x = layers.Dense(128, activation="relu")(latent_inputs)**: Dense layer to expand latent vector.
- **x = layers.Dense(28 * 28, activation="sigmoid")(x)**: Output layer to generate 784 pixels with values between 0 and 1.

```
latent_inputs = keras.Input(shape=(latent_dim,))

x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)

x = layers.Reshape((7, 7, 64))(x)

x = layers.Conv2DTranspose(128, 3, activation="relu",
                           strides=2, padding="same")(x)

x = layers.Conv2DTranspose(64, 3, activation="relu",
                           strides=2, padding="same")(x)
```

```
decoder_outputs = layers.Conv2DTranspose(
    1, 3, activation="sigmoid", padding="same")(x)

decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")

decoder.summary()
```

Output:

Model: "decoder"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 2)	0
dense_2 (Dense)	(None, 3136)	9,408
reshape (Reshape)	(None, 7, 7, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 128)	73,856
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 64)	73,792
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	577

Total params: 157,633 (615.75 KB)
Trainable params: 157,633 (615.75 KB)
Non-trainable params: 0 (0.00 B)

Summary

Step 5: Defining the VAE Model

Combine encoder and decoder into the VAE model and define the custom training step including reconstruction and KL-divergence losses.

- **self.loss_fn = keras.losses.BinaryCrossentropy(from_logits=False)**: Set reconstruction loss as binary cross-entropy.
- **with tf.GradientTape() as tape**: Record operations for gradient calculation.
- **kl_loss = -0.5 * tf.reduce_mean(1 + log_var - tf.square(mean) - tf.exp(log_var))**: Calculate KL divergence loss.

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = keras.metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")
```

```
    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker,
        ]

    def train_step(self, data):
        with tf.GradientTape() as tape:
            mean, log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    keras.losses.binary_crossentropy(data, reconstruction),
                    axis=(1, 2),
                )
            )
            kl_loss = -0.5 * (1 + log_var - tf.square(mean) - tf.exp(log_var))
            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
            total_loss = reconstruction_loss + kl_loss
        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
        }
```

Step 6: Training the VAE

Load the Fashion-MNIST dataset and train the model for 10 epochs.

- **x_train = np.expand_dims(x_train, -1):** Add channel dimension to training images.
- **x_test = x_test.astype("float32") / 255.0**: Normalize test images.
- **x_test = np.expand_dims(x_test, -1)**: Add channel dimension to test images.

```
(x_train, _), (x_test, _) = keras.datasets.fashion_mnist.load_data()
fashion_mnist = np.concatenate([x_train, x_test], axis=0)
fashion_mnist = np.expand_dims(fashion_mnist, -1).astype("float32") / 255
```

```
vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())
vae.fit(fashion_mnist, epochs=10, batch_size=128)
```

Step 7: Displaying Sampled Images

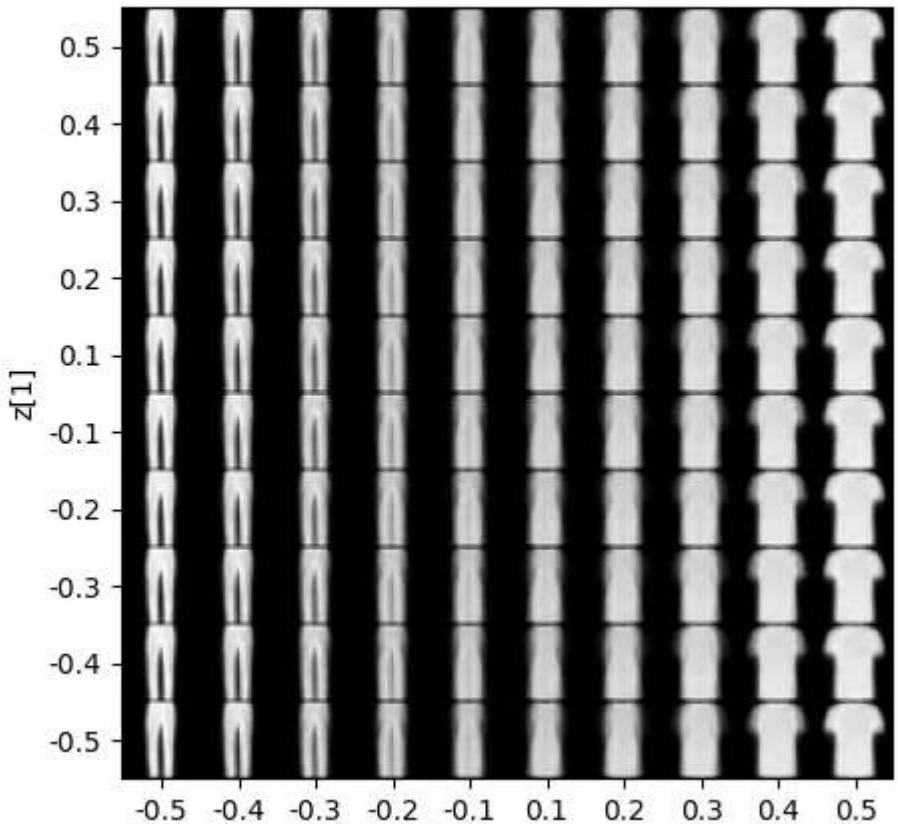
Generate new images by sampling points from the latent space and display them.

- **z_sample = np.array([[xi, yi]])**: Create latent vector from grid point.
- **x_decoded = decoder.predict(z_sample)**: Decode latent vector to image.

```
import matplotlib.pyplot as plt

def plot_latent_space(vae, n=10, figsize=5):

    img_size = 28
    scale = 0.5
    figure = np.zeros((img_size * n, img_size * n))

    grid_x = np.linspace(-scale, scale, n)
    grid_y = np.linspace(-scale, scale, n)[::-1]

    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):
            sample = np.array([[xi, yi]])
            x_decoded = vae.decoder.predict(sample, verbose=0)
            images = x_decoded[0].reshape(img_size, img_size)
            figure[
                i * img_size : (i + 1) * img_size,
                j * img_size : (j + 1) * img_size,
            ] = images

    plt.figure(figsize=(figsize, figsize))
    start_range = img_size // 2
    end_range = n * img_size + start_range
    pixel_range = np.arange(start_range, end_range, img_size)
    sample_range_x = np.round(grid_x, 1)
    sample_range_y = np.round(grid_y, 1)
    plt.xticks(pixel_range, sample_range_x)
    plt.yticks(pixel_range, sample_range_y)
    plt.xlabel("z[0]")
    plt.ylabel("z[1]")
    plt.imshow(figure, cmap="Greys_r")
```

```
    plt.show()


plot_latent_space(vae)
```

Output:



Step 8: Displaying Latent Space Clusters

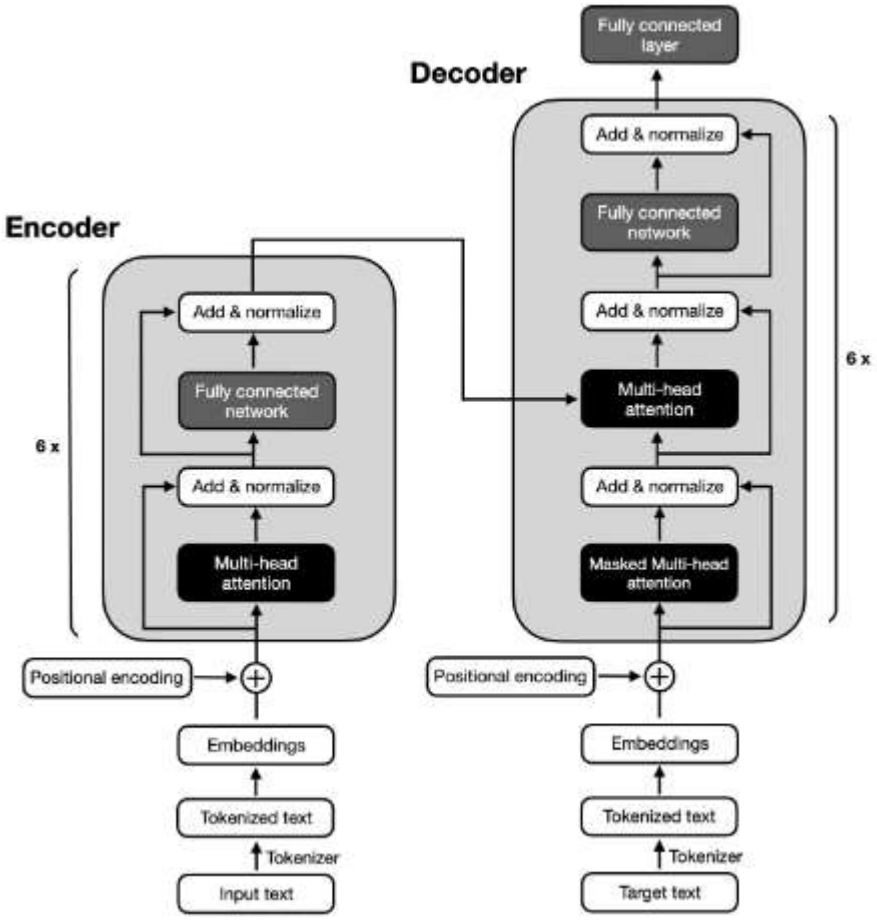
Encode the test set images and plot their positions in latent space to visualize clusters.

- **mean, _, _ = encoder.predict(x_test)**: Encode test images to latent mean vectors.

```
def plot_label_clusters(encoder, decoder, data, test_lab):
    z_mean, _, _ = encoder.predict(data)
    plt.figure(figsize=(12, 10))
    sc = plt.scatter(z_mean[:, 0], z_mean[:, 1], c=test_lab)
    cbar = plt.colorbar(sc, ticks=range(10))
    cbar.ax.set_yticklabels([labels.get(i) for i in range(10)])
    plt.xlabel("z[0]")
    plt.ylabel("z[1]")
    plt.show()


labels = {0: "T-shirt / top",
```

```
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle boot"}

(x_train, y_train), _ = keras.datasets.fashion_mnist.load_data()
x_train = np.expand_dims(x_train, -1).astype("float32") / 255
plot_label_clusters(encoder, decoder, x_train, y_train)
```

Encoder-Decoder architectures



Encoder–Decoder architecture is a deep learning framework mainly used in **sequence-to-sequence (Seq2Seq)** problems where the input and output are sequences and may have different lengths. It is widely used in **Natural Language Processing (NLP)**, speech processing, and generative AI applications.

The Encoder–Decoder model consists of two main components:

1. Encoder

The encoder processes the input sequence and converts it into a fixed-length context vector (also called thought vector).

- Takes input sequence: X=(x1,x2,x3,…,xn)
- Encodes it into a hidden representation.
- Final hidden state summarizes the entire input sequence

2. Decoder

The decoder takes the context vector and generates the output sequence step-by-step.

- Produces output sequence: Y=(y1,y2,y3,…,ym)
- Each output token depends on:
  - Previous output token
  - Context vector
  - Decoder hidden state

Working Process

1. Input sentence is fed into the encoder.
2. Encoder processes tokens sequentially using RNN/LSTM/GRU.
3. Final hidden state becomes the context vector.
4. Decoder uses this vector to generate output one token at a time.
5. Process continues until an end-of-sequence token is generated

Mathematical Representation

Encoder hidden state:

$$h_t = f(x_t, h_{t-1})$$

Context vector:

$$c = h_n$$

Decoder hidden state:

$$s_t = f(y_{t-1}, s_{t-1}, c)$$

Output probability:

$$P(y_t) = softmax(W s_t)$$

Types of Encoder–Decoder Architectures

(A) RNN-based Encoder–Decoder

- Uses RNN, LSTM, or GRU.
- Problem: Information bottleneck due to fixed context vector.

(B) Attention-based Encoder–Decoder

Introduced to solve fixed-length vector problem.

- Decoder focuses on different parts of input at each step.
- Improves translation quality.
- Example: Attention mechanism introduced by Dzmitry Bahdanau and colleagues.

(C) Transformer-based Encoder–Decoder

Uses self-attention instead of recurrence.

- Parallel processing
- Better long-range dependency handling
- Example: Google introduced Transformer architecture.

Famous models:

- OpenAI models (e.g., GPT variants – decoder-only)
- Google BERT (encoder-only)

Stable diffusion model

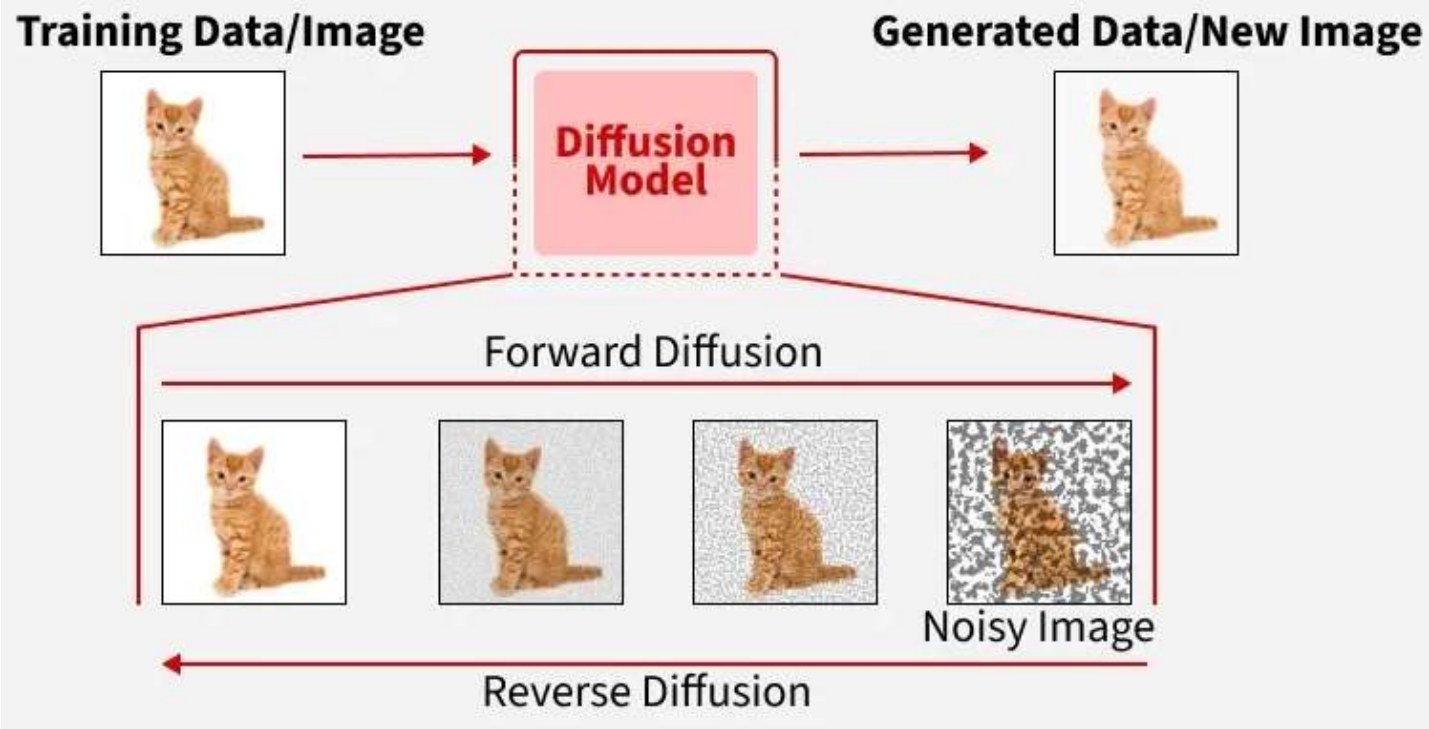
Stable Diffusion is a technique used in generative artificial intelligence, particularly in the context of image generation. It's an extension of the diffusion probabilistic model, which is a generative model used for image generation tasks. The diffusion model essentially learns to generate images by iteratively demising a random noise input. Stable Diffusion builds upon this by introducing stability mechanisms to improve training and sample quality. This stability is achieved through various means, such as regularization techniques, architectural modifications, or algorithmic improvements

The Stable Diffusion Model is a mathematical model used in physics and statistical mechanics to describe the process of diffusion in complex systems. Diffusion is the movement of particles or molecules from an area of higher concentration to an area of lower concentration, driven by random thermal motion. In many systems, especially those with disorder or heterogeneity, standard diffusion models like Fick's laws may not adequately describe the behavior observed. The Stable Diffusion Model addresses this by considering scenarios where the underlying distribution of displacements of diffusing particles has heavy tails, meaning that rare, extreme events occur more frequently than would be predicted by a normal distribution.

The Stable Diffusion Model is based on stable distributions, which are a class of probability distributions characterized by heavy tails. These distributions are defined by four parameters: the stability index α, skewness β, scale parameter γ, and location parameter

Stable Diffusion is a technique in the field of generative artificial intelligence (AI) that aims to generate high-quality images. It is an extension of diffusion probabilistic models, which are a class of generative models used for image generation. Stable Diffusion is important because it addresses some of the limitations of earlier diffusion models, particularly in generating high-resolution and high-fidelity images.

Diffusion models are a type of generative AI that create new data like images, audio or even video by starting with random noise and gradually turning it into something meaningful. They work by simulating a diffusion process where data is slowly corrupted by noise during training and then learning to reverse this process step by step. By doing so the model learns how to generate high quality samples from scratch



- Diffusion models are generative models that learn to reverse a diffusion process to generate data. The diffusion process involves gradually adding noise to data until it becomes pure noise.
- Through this process a simple distribution is transformed into a complex data distribution in a series of small incremental steps.
- Essentially these models operate as a reverse diffusion phenomenon where noise is introduced to the data in a forward manner and removed in a reverse manner to generate new data samples.
- By learning to reverse this process diffusion models start from noise and gradually denoise it to produce data that closely resembles the training examples.

1. **Forward Diffusion Process**: This process involves adding noise to the data in a series of small steps. Each step slightly increases the noise, making the data progressively more random until it resembles pure noise.
2. **Reverse Diffusion Process**: The model learns to reverse the noise-adding steps. Starting from pure noise, the model iteratively removes the noise, generating data that matches the training distribution.

3. **Score Function**: This function estimates the gradient of the data distribution concerning the noise. It helps guide the reverse diffusion process to produce realistic samples.

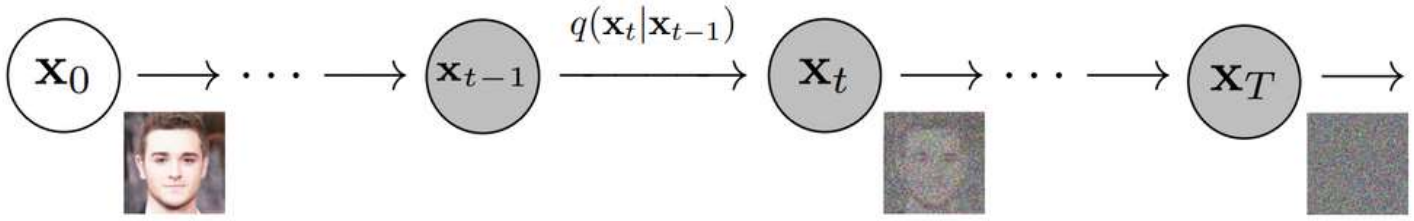
Architecture of Diffusion Models

The architecture of diffusion models typically involves two main components:

1. Forward Diffusion Process
2. Reverse Diffusion Process

1. Forward Diffusion Process

In this process noise is incrementally added to the data over a series of steps. This is akin to a Markov chain where each step slightly degrades the data by adding Gaussian noise.



Mathematically, this can be represented as

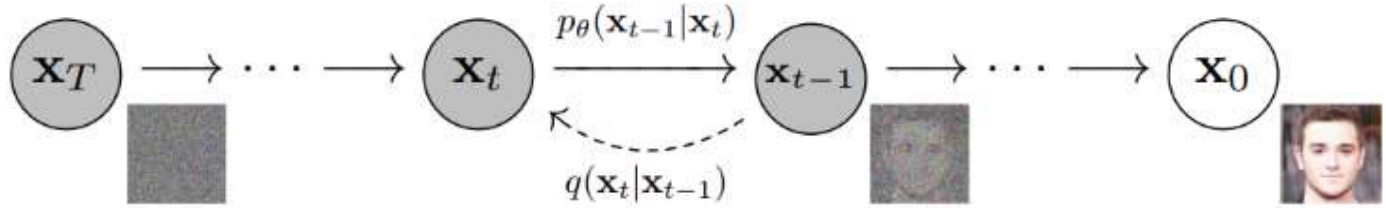
$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$$

where,

- x_t is the noisy data at step t
- α_t controls the amount of noise added.

2. Reverse Diffusion Process

The reverse process aims to reconstruct the original data by denoising the noisy data in a series of steps reversing the forward diffusion.



Reverse Diffusion Process

This is typically modelled using a neural network that predicts the noise added at each step:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_\theta(x_t, t))$$

where,

- $\mu\theta$ and $\sigma\theta$ are learned parameters.

Working Principle of Diffusion Models

During training the model learns to predict the noise added at each step of the forward process. This is done by minimizing a loss function that measures the difference between the predicted and actual noise.

Forward Process (Diffusion)

- The forward process involves gradually corrupting the data 0x_0 with Gaussian noise over a sequence of time steps
- Let x_t represent the noisy data at time step t. The process is defined as:

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$$

- where β_t is the noise schedule that controls the amount of noise added at each step and ϵ is is Gaussian noise.
- As t increases, x_t becomes more noisy until it approximates a Gaussian distribution.

Reverse Process (Denoising)

- The reverse process aims to reconstruct the original data x_0 from the noisy data x_T at the final time step T.
- This process is modelled using a neural network to approximate the conditional probability $p_\theta(x_{t-1}|x_t)$.
- The reverse process can be formulated as:

$$x_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\beta_t}}\epsilon_\theta(x_t, t)\right)$$

- where ϵ_θ is a neural network parameterized by θ that predicts the noise.

Introduction to Transformer-based Image Generation

Transformer-based image generation is a modern approach in **Generative AI** that uses the **Transformer architecture** (originally developed for Natural Language Processing) to generate images instead of text. Unlike traditional CNN-based models or GANs, transformers rely mainly on **self-attention mechanisms** to understand global relationships in data.

The transformer architecture was first introduced in the paper **"Attention Is All You Need"** by Ashish Vaswani and his team in 2017. Initially designed for language translation, transformers are now widely used in image generation, video generation, and multimodal AI systems.

Step 1: Image Tokenization

An image is divided into small patches (e.g., 16×16).
Each patch is flattened and converted into a vector (token).
This concept is inspired by **Vision Transformers (ViT)**.

Step 2: Embedding

Each image patch is converted into an embedding vector.
Positional encoding is added to maintain spatial information.

Step 3: Self-Attention Mechanism

The transformer applies **Multi-Head Self-Attention**:

- Each patch attends to every other patch.
- Learns global structure.
- Captures relationships between distant regions of the image.

Step 4: Autoregressive or Diffusion-Based Generation

Transformer-based generation can follow different strategies:

(a) Autoregressive Models

Generate image tokens one by one (similar to text generation).
Example:

- OpenAI developed DALL·E, which generates images from text descriptions.

(b) Diffusion + Transformer

Transformers are used inside diffusion models to improve quality.
Example:

- OpenAI's Sora (video generation).
- Stability AI's Stable Diffusion (uses attention-based architecture).

Popular Transformer-Based Image Generation Models

1. **Vision Transformer (ViT)** – Image understanding.
2. **Image GPT (iGPT)** – Treats image pixels as tokens.
3. **DALL·E** – Text-to-image generation.
4. **Swin Transformer** – Hierarchical transformer for vision tasks.
5. Latent Diffusion Models with transformers.

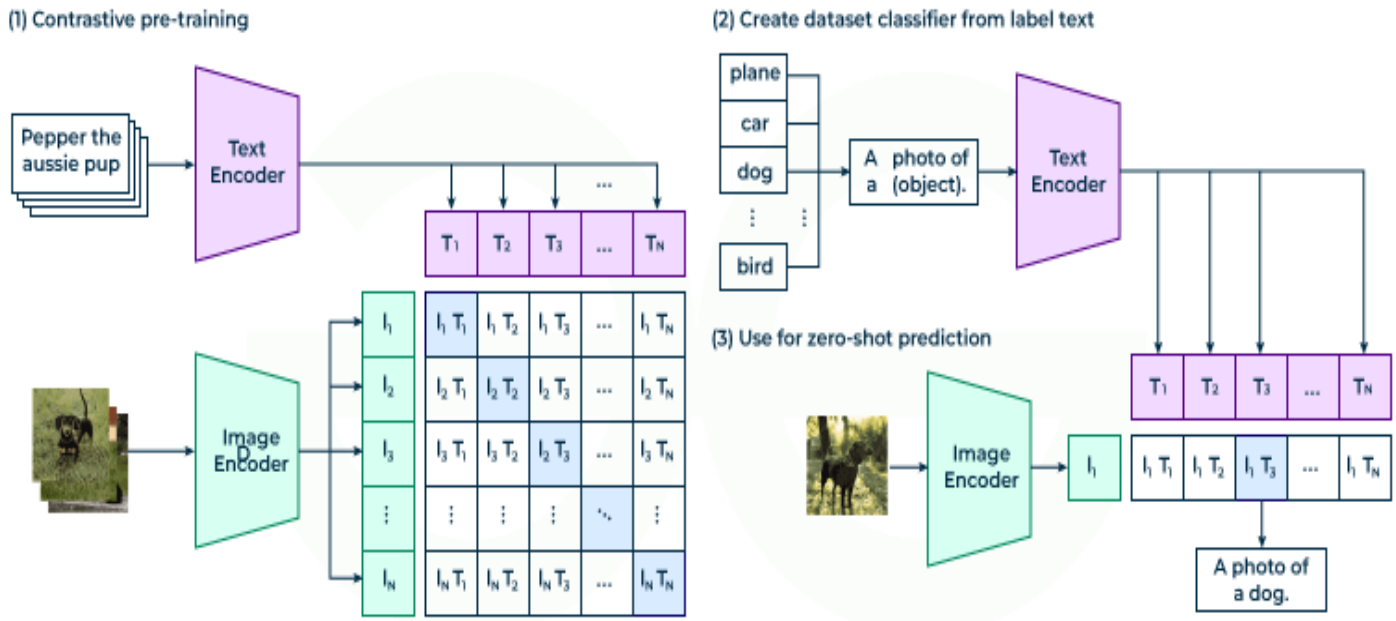
Transformer-based image generation represents a significant advancement in Generative AI. By leveraging **self-attention mechanisms**, transformers can model global relationships in images more effectively than CNN-based models like GANs and VAEs.
With the success of models like **DALL·E** and **Stable Diffusion**, transformers have become a foundational technology for modern image generation systems.

CLIP

In modern **Transformer-based image generation**, **CLIP (Contrastive Language–Image Pretraining)** plays a crucial role in aligning text and images. CLIP was developed by OpenAI and introduced in 2021 by researchers including Alec Radford. CLIP is not itself a generator. Instead, it acts as a **guidance model** that connects text and images in a shared representation space, which helps transformer-based generative models produce images that match textual descriptions

CLIP or Contrastive Language-Image Pretraining is an advanced AI model developed by OpenAI and UC Berkeley. It has the unique ability to understand and relate both textual descriptions and images. It uses a novel training method that contrasts pairs of images and text which makes it highly useful tool for various real-world applications. In this article, we'll see the fundamentals of how CLIP works, its innovative approach and its other core concepts.

Let us understand the architectural details of CLIP. Below is the architecture of the CLIP neural network:



1. Text Encoder

CLIP uses a Transformer-based model (similar to the model in the Attention is All You Need paper). This model converts text into embeddings, dense vectors that capture the meaning of the text. Its text encoder is a 63M-parameter model with 12 layers and 8 attention heads.

2. Image Encoder

For the image encoder, it experimented with both ResNet and Vision Transformers (ViT). At last ViT was chosen due to its superior performance in processing images. This encoder transforms images into embeddings that capture the image's key features.

3. Dataset

CLIP was trained on a massive dataset of 400 million image-text pairs sourced from the web. The team focused on using words that appeared at least 100 times in the English Wikipedia which ensures that 500,000 words were covered. This dataset called **WebImageText (WIT)** is important to CLIP's ability to generalize to various visual and textual concepts.

4. Training Objective

CLIP's goal is to align text and image embeddings which ensures that correct pairs of image and text are similar while incorrect pairs are not.

- **Cosine Similarity:** It maximizes similarity for matching pairs and minimizes it for non-matching pairs.
- **Training from Scratch:** Both image and text encoders are trained from the ground up, without pre-trained weights.
- **Projection:** The embeddings generated by the image and text encoder are projected into a shared space with the same dimensionality to allow for effective comparison.
- **Contrastive Learning:** During training, it distinguishes between correct and incorrect image-text pairs which optimizes the model to correctly identify matches.
- **Loss Function:** Cross-entropy loss is used to adjust the model, maximizing similarity for correct pairs and minimizing it for incorrect ones.
- **Inference:** After training, it calculates similarity scores between image-text pairs to determine relevance.

CLIP's Unique Features

CLIP stands out from traditional models because of these key features:

- **Multimodal Training:** Traditional models process images and text separately. It, however, trains on both images and text at the same time which helps it to learn their relationship.
- **Zero-Shot Learning**: CLIP doesn't need to be retrained for new categories. After the initial training, it can classify images into any category even ones it hasn't seen before. This zero-shot learning is useful when retraining is not practical.
- **Self-Supervised Learning**: CLIP doesn't need explicit labels for every image or category. It learns by distinguishing between correct and incorrect image-text pairs which makes it a self-supervised model.

Visual Transformers ViT- Dall-E2 and Dall-E3

Transformer-based models have significantly improved image understanding and generation. Important developments include **Vision Transformers (ViT)** for image understanding and advanced text-to-image systems such as **DALL·E 2** and **DALL·E 3** developed by OpenAI.

Vision Transformer (ViT) is a deep learning architecture that applies the Transformer model to images. Instead of relying on convolutions, ViTs use self-attention to capture relationships across all image patches, enabling a global understanding of the image. This approach ha
s achieved state-of-the-art results in various computer vision tasks.

- Uses self-attention to model global dependencies between image patches.
- Unlike CNNs, it does not rely on convolution operations for feature extraction.
- Demonstrates strong performance in image classification, object detection and segmentation.

DALL –E2

It is an image-generation AI that can create detailed and creative images based on descriptions you give it. For example, if you prompt it with something like *"a futuristic city floating above the clouds in watercolor style"*, it will generate an image matching that description.

There are three main components of DALLE-2:

1. CLIP: Clip is an open-source model and it tells whether a text embedding matches the image embedding or not.
2. Diffusion Prior (based on transformer): For generating clip image embeddings based on clip text embeddings.
3. Decoder (based on diffusion model): For generating image.

Below is the overall architecture as presented in the original paper. Let us discuss each of the components in detail.

Dall-E 2 Architecture

There are three main components of DALLE-2:

CLIP: Clip is an open-source model and it tells whether a text embedding matches the image embedding or not.

Diffusion Prior (based on transformer): For generating clip image embeddings based on clip text embeddings.

Decoder (based on diffusion model): For generating image.

Below is the overall architecture as presented in the original paper. Let us discuss each of the components in detail.

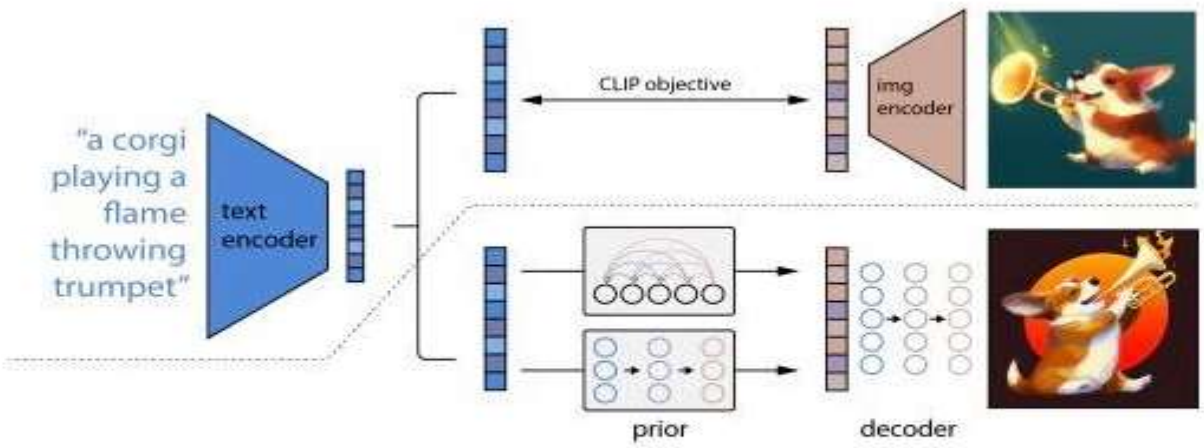


Figure 2: A high-level overview of unCLIP. Above the dotted line, we depict the CLIP training process, through which we learn a joint representation space for text and images. Below the dotted line, we depict our text-to-image generation process: a CLIP text embedding is first fed to an autoregressive or diffusion prior to produce an image embedding, and then this embedding is used to condition a diffusion decoder which produces a final image. Note that the CLIP model is frozen during training of the prior and decoder.

1. Contrastive Language-Image Pre-training (CLIP)

The component shown above the dotted line is the clip component. CLIP is trained first. The objective of CLIP is to align text embeddings and image embeddings. CLIP tells how much a given text relates to an image.Note - It does not tell the caption for an image. It only whether a given description is a good fit or not for a given image
CLIP is trained as per the below process.
All image and their true captions or descriptions are encoded. There is a separate encoder for images and a separate encoder for text.
The cosine similarity between the encoded vectors of the image and text description is calculated.
The training objective is to maximize the similarity between the correct pair of image and text embeddings and minimize the similarity between incorrect image and text embeddings.
During inference time the embedding of image and text description is obtained through the trained image and text encoder and a similarity score is calculated which indicates the relevance of text descriptionconcerningo the image
Below is the architecture of CLIP as per the original paper

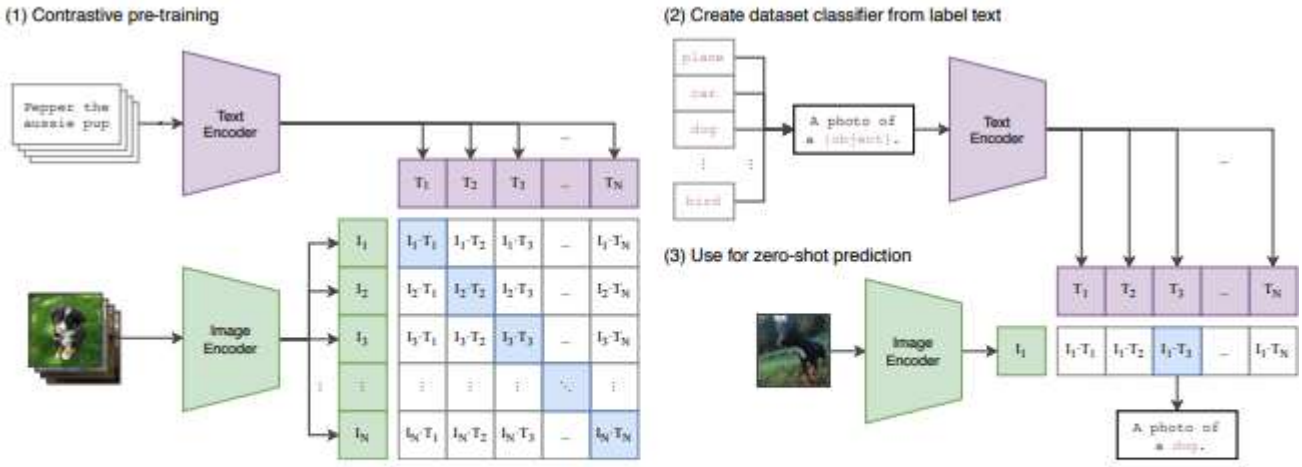


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

2. Diffusion Prior

After the training of CLI, P the diffusion prior and image decoder are trained. The objective of diffusion prior is to take text embedding and find its associated CLIP image embeddings. Too so it takes help for the CLIP model trained in step 1.

The diffusion prior is a decoer only transformed with a causal attention mask. The input consists of 4 vectors.

1. Noise clip image embedding
2. Diffusiontimestepp
3. Encoded text
4. CLIP text embedding

- For a given correct text image pair we find its CLIP image embedding and CLIP text embedding
- Noise is added to the CLIP image embedding. The amount of noise added depends on the diffusion timestep. As we proceed further and further more noise is added
- The purpose of the diffusion prior is to predict the CLIP image embeddings.

- The MSE loss between the original CLIP embedding and predicted CLIP image embedding is used as the loss function

3. Image Decoder

The purpose of the image decoder is to use the CLIP image embedding predicted by the diffusion prior and generate an image based on this. The diffusion model of the image decoder is based on the paper GLIDE: Towards Photorealistic Image Generation and Editing with text-Guided Diffusion Models. The GLIDE paper combined the diffusion process with UNET architecture.

UNET Architecture:

- The left part of the U-NET architecture consists of a series of convolutional and pooling layers, which function as an encoder. This part captures the contextual information in the input image.
- The right part of the U-Nent consists of a series of upsampling and convolutional layers, forming the decoder
- The U-Net architecture consists of skip connections. These connections directly link the corresponding encoder and decoder layers to enable the network to use both high-level spatial information and low-level spatial information.

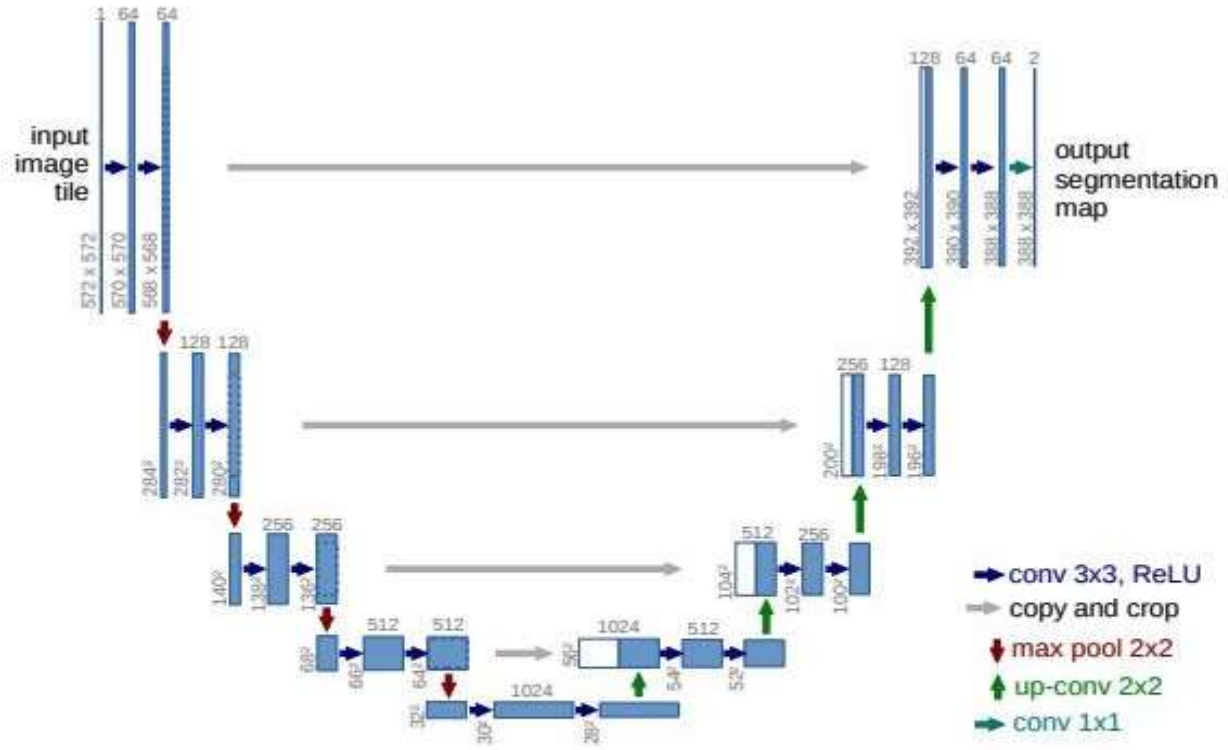


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Dall E3

DALL·E 3 is an advanced text-to-image generative AI model developed by OpenAI. It is the third major version in the DALL·E series, designed to generate highly detailed and contextually accurate images from natural language descriptions. It represents a significant improvement over DALL·E 2 in terms of prompt understanding, image quality, safety, and integration with conversational AI systems.

DALL·E 3 is tightly integrated with ChatGPT, enabling users to generate images directly through conversational prompts. This integration allows automatic prompt refinement, making it easier for users to create high-quality images even with simple descriptions.

The main objectives of DALL·E 3 are:

1. To generate high-resolution, realistic, and artistic images from textual descriptions.
2. To improve understanding of complex and long prompts.
3. To reduce prompt engineering effort required by users.
4. To ensure safer and responsible image generation.
5. To support creative, educational, and professional applications
6. **High Prompt Adherence:** DALL-E 3 excels at translating detailed descriptions into images, rendering specific details like text, faces, and hands with high accuracy.
7. **ChatGPT Integration:** It uses ChatGPT to rewrite user prompts, enhancing them for better composition and detail.
8. **Iterative Editing:** Users can refine images through conversation (e.g."make it more old-fashioned").
9. **Aspect Ratios:** It supports landscape (1024x1792) and portrait (1792x1024) orientations, along with the default square (1024x1024).
10. **Safety & Ethics:** It includes restrictions against generating images in the style of living artists, public figures, or offensive content.
11. **Versatility:** Suitable for creating logos, product designs, and artistic illustrations

To get the best results from DALL·E 3 keep these tips in mind:

- **Detailed Prompts: It** understands prompts and some people give short forms and expects quality images to get the best image users need to give long and descriptive prompts.
- **Number and positions: It** understands and keeps track of information i.e people can give a proper number or position of an object in an image.
- **Making Variations:** People can ask DALL.E 3 to make a small change or any type of change in their image and this tool will make it within seconds.

We can use **DALL-E 3** to generate images effortlessly from simple descriptions and can make further modifications in that image

GPT-4V

GPT-4V (Generative Pre-trained Transformer 4 with Vision) is an advanced **multimodal large language model** developed by OpenAI. It extends the capabilities of GPT-4 by enabling the model to understand and process **both text and images** as input.

Unlike traditional language models that work only with text, GPT-4V can analyze images, interpret visual content, and generate meaningful textual responses based on visual inputs. This makes it a powerful system for tasks involving image understanding, reasoning, and multimodal interaction

GPT-4V (Generative Pre-trained Transformer 4 with Vision) is an advanced **multimodal large language model** developed by OpenAI. It extends the capabilities of GPT-4 by enabling the model to understand and process **both text and images** as input.

Unlike traditional language models that work only with text, GPT-4V can analyze images, interpret visual content, and generate meaningful textual responses based on visual inputs. This makes it a powerful system for tasks involving image understanding, reasoning, and multimodal interaction

GPT-4V is based on the **Transformer architecture** with multimodal extensions.

The architecture consists of:

1. **Image Encoder**
    - Converts image input into numerical representations (embeddings).
    - Extracts visual features such as objects, shapes, text, and spatial relationships.
2. **Text Encoder**
    - Processes textual prompts using transformer layers.
    - Generates contextual embeddings for text.
3. **Multimodal Fusion Layer**
    - Combines image embeddings and text embeddings.
    - Enables cross-modal attention between visual and textual data.
4. **Decoder / Output Layer**
    - Generates coherent textual responses based on combined representations.

The model uses **self-attention and cross-attention mechanisms** to connect visual and textual features effectively.

4. Key Features of GPT-4V

1. **Image Understanding**
    - Describes objects, scenes, and activities in images.
    - Detects text inside images (OCR-like capability).
2. **Visual Reasoning**
    - Explains graphs, charts, and diagrams.
    - Solves math problems shown in images.
    - Interprets complex layouts.
3. **Contextual Awareness**
    - Maintains conversation context.
    - Connects image details with previous dialogue.
4. **Multilingual Capability**

- Understands and responds in multiple languages.

5. **Improved Accuracy**
   - Better reasoning and fewer hallucinations compared to earlier multimodal systems.

5. Working of GPT-4V

The working process can be explained in three stages:

(i) Input Processing

- User provides an image along with a textual prompt.
- Image is converted into feature vectors.
- Text is tokenized and converted into embeddings.

(ii) Multimodal Understanding

- The system applies attention mechanisms.
- It aligns visual regions with textual queries.
- It performs reasoning across both modalities.

(iii) Response Generation

- The model predicts the next words sequentially.
- Generates explanation, description, or solution in text form.
-

Key Issues in Image Generation Models:

- **Mode Collapse:** This occurs when the generator in a Generative Adversarial Network (GAN) identifies a limited, specific set of images that trick the discriminator and repeatedly produces them, ignoring the diversity of the training data. This results in a lack of variety, often seen in medical imaging where only one type of organ, lesion, or feature is generated regardless of input.
- **Training Instability (GANs):** GAN training is notorious for being unstable. It involves a "cat and mouse" game between the generator and discriminator, which often leads to non-convergence.
  - **Vanishing Gradients:** When the discriminator becomes too effective, it provides no useful feedback to the generator, causing training to stall.
  - **Exploding Gradients:** These can cause massive, destructive weight updates.
- **Model Collapse (Recursive Training):** A distinct issue where models trained on data generated by previous AI iterations show degradation in performance over time, resulting in "model autophagy disorder" (MAD), or a "mad cow" effect of increasing nonsense outputs.
- **Diversity vs. Fidelity Trade-off:** Models often struggle to balance generating highly realistic images (high fidelity) while still producing a wide range of, or diverse, images.

Wiley Online Library +7

Solutions and Mitigation Strategies:

- **Wasserstein Loss (WGANs):** Uses Wasserstein distance instead of standard loss, providing better, more consistent gradients to stabilize training and mitigate mode collapse.
- **Unrolled GANs:** Allows the generator to look ahead at the future state of the discriminator, reducing the likelihood of falling into a single mode.
- **Alternative Architectures:** Using Diffusers or VAEs (though VAEs often suffer from blurriness due to averaging in latent space).
- **Regularization:** Adding noise to the discriminator's input or limiting weights to improve training stability

Issues in Stability AI Models:

- **Prompt Understanding & Component Inclusion:** Models struggle to render all elements when multiple objects are requested in a single prompt, resulting in a 8.53% drop in Component Inclusion Score (CIS) per component.
- **Anatomical and Structural Incoherence:** Despite improvements, models often generate distorted, warped, or inaccurate human features (e.g., extra fingers, misshapen faces) and, at times, fail to maintain realistic lighting or texture.
- **Text Rendering Limitations:** Rendering legible, accurate text within images remains a major challenge, with words often appearing as blurry, distorted, or misspelled, even in newer iterations.
- **Safety and Bias:** Research on Stable Diffusion models reveals that many models lack proper safety filters, leading to the generation of harmful, unethical, or highly biased content.
- **Computational and Temporal Efficiency:** While more efficient than earlier GANsStable Diffusion faces challenges with high inference times, making real-time generation difficult.
    - **Spatial Reasoning:** The models struggle with spatial relationships (e.g."a red cube on top of a blue sphere"), often confusing the placement of objects